

0. Dokumentum adatai

1. Bevezetés

- 1.1. A Projekt Célja
- 1.2. Főbb Funkciók
- 1.3. Technológiai Stack
- 1.4. Fogalomtár (Glossary)

2. Rendszerarchitektúra

- 2.1. Magas Szintű Architektúra
- 2.2. Komponensek
- 2.3. Adatbázis Séma

3. Backend Architektúra

- 3.1. Backend funkciók és felelősségek
 - 3.1.1. Felhasználókezelés és autentikáció
 - 3.1.2. Jogosultságok és szerepkörök
 - 3.1.3. Jegyzetek kezelése
 - 3.1.4. Fájlfeltöltés és validáció
 - 3.1.5. Közösségi funkciók
 - 3.1.6. Jelentés és moderáció
 - 3.1.7. Profil és testreszabás
 - 3.1.8. Üzenetek, barátok, értesítések
 - 3.1.9. Csoport funkciók
 - 3.1.10. Lokalizáció
 - 3.1.11. Adatbázis hozzáférési segédek és biztonság
- 3.2. Adatbázis kapcsolat
- 3.3. Fájlkezelés

4. Fejlesztői Dokumentáció

- 4.1. Fejlesztői Környezet Beállítása
- 4.2. Verziókezelési Stratégia
 - 4.2.1. Verziószám felépítése
 - 4.2.2. Fejlesztési (beta) állapot jelölése
 - 4.2.3. Átmenet végleges verzióra
 - 4.2.4. Verziózás a CHANGELOG-ban
 - 4.2.5. Dátumformátum szabályok
 - 4.2.6. Verziókezelés és Git kapcsolata
 - 4.2.7. Összefoglalás
- 4.3. FájlStruktúra

- 4.4. [Fejlesztői eszközök, script-ek és refaktorok](#)
- 4.5. [Konfiguráció](#)
- 4.6. [Fő folyamatok](#)
- 4.7. [Security checklist \(dev\)](#)
- 4.8. [Debug / logging](#)
- 4.9. [Változásnapló \(Changelog\)](#)
- 4.10. [Dokumentáció karbantartás](#)
- 4.11. [Oldalak részletes referenciája](#)
 - 4.11.1. [index.php](#)
 - 4.11.2. [reglog.php](#)
 - 4.11.3. [upload.php](#)
 - 4.11.4. [note.php](#)
 - 4.11.5. [admin_panel.php](#)
 - 4.11.6. [group.php](#)
 - 4.11.7. [create_group.php](#)
 - 4.11.8. [favorites.php](#)
 - 4.11.9. [forgotpass.php](#)
 - 4.11.10. [2fa.php](#)
 - 4.11.11. [groups.php](#)
 - 4.11.12. [messages.php](#)
 - 4.11.13. [notify.php](#)
 - 4.11.14. [premium.php](#)
 - 4.11.15. [payment_demo.php](#)
 - 4.11.16. [privacy.php](#)
 - 4.11.17. [terms.php](#)
 - 4.11.18. [profile.php](#)
 - 4.11.19. [reg-ver.php](#)
 - 4.11.20. [search.php](#)
 - 4.11.21. [note_stats.php](#)
 - 4.11.22. [messages](#) (kiegészítés)
 - 4.11.23. [assets/php/db.php](#)
 - 4.11.24. [assets/php/functions.php](#)
 - 4.11.25. [assets/php/group_actions.php](#)
 - 4.11.26. [assets/php/group_init.php](#)
 - 4.11.27. [assets/php/loadmessages.php](#)
 - 4.11.28. [assets/php/download.php](#)
 - 4.11.29. [assets/php/report.php](#)
 - 4.11.30. [assets/php/premium.php](#)
 - 4.11.31. [assets/php/accept_friend.php](#)
 - 4.11.32. [assets/php/add_friend.php](#)

- 4.11.33. [assets/php/ads.php](#)
- 4.11.34. [assets/php/delete.php](#)
- 4.11.35. [assets/php/findanything.php](#)
- 4.11.36. [assets/php/findtag.php](#)
- 4.11.37. [assets/php/footer.php](#)
- 4.11.38. [assets/php/kereso_tag.php](#)
- 4.11.39. [assets/php/lang.php](#)
- 4.11.40. [assets/php/logout.php](#)
- 4.11.41. [assets/php/mail-2fa.php](#)
- 4.11.42. [assets/php/mail-regver.php](#)
- 4.11.43. [assets/php/navbar.php](#)

0. Dokumentum adatai

Tulajdonság	Érték
Projekt neve	Jegyzetár
Dokumentum típusa	Fejlesztői + felhasználói dokumentáció
Dokumentum azonosító	JEGYZETAR-DOKU-2026-01
Verzió	1.0.0
Kiadás dátuma	TBD
Utolsó frissítés	2026-01-15
Állapot	Draft
Célközönség	Diákok, tanárok, fejlesztők
Repository	doomhyena/jegyzetar.eu-src
Célplatform	Web (reszponzív)
Min. PHP	8.2+
Támogatott böngészők	Chrome, Firefox, Edge (friss)
Kapcsolat	info@jegyzetar.hu

Verziózás: SemVer (major.minor.patch)

Megjegyzés: A dokumentáció a projekt aktuális állapotát tükrözi, a változásokat a CHANGELOG rögzíti.

1. Bevezetés

1.1. A Projekt Célja

A **Jegyzetár** egy modern, közösségi alapú, oktatássegítő platform, amely lehetővé teszi a diákok számára, hogy egyszerűen és biztonságosan megosszák egymással jegyzeteiket, segédanyagaikat és tanulást támogató dokumentumaikat. A projekt célja, hogy:

- **Kik használják?**
Diákok, tanárok és oktatók, akik megosztanák egymással az oktatási anyagokat.
- **Gyors és kényelmes fájlmegosztást biztosítson tanulóknak**
 - A felhasználók könnyedén feltölthetnek, kereshetnek és letölthetnek jegyzeteket
 - A platform reszponzív kialakítása biztosítja a mobilbarát használatot
 - Az egyszerű kezelőfelület csökkenti az informatikai tudás iránti igényt
- **Rendszerezett és kereshető tananyagbázist hozzon létre**
 - Tantárgyak, évfolyamok és dokumentumtípusok szerint kategorizál
 - Kulcsszavas keresés és szűrés segíti a gyors anyagkeresést
 - Előnézeti kép vagy rövid leírás segít a tartalom gyors azonosításában
- **Támogassa a közösségi tanulást**
 - A felhasználók értékelhetik, kommentelhetik a jegyzeteket
 - A rendszer kiemeli a legnépszerűbb vagy legjobbra értékelt anyagokat

1.2. Főbb Funkciók

A Jegyzetár rendszer az alábbi kulcsfunkciókat biztosítja:

Fájlfeltöltés és -kezelés

- Jegyzetek és segédanyagok feltöltése PDF, DOCX, MP4 formátumban
- Automatikus kategorizálás tantárgy és dokumentumtípus szerint
- Előnézeti kép generálása (PDF első oldal, képek)

Felhasználói felület

- Regisztráció és bejelentkezés (alap és "OAuth")
- Egyéni irányítópult a feltöltések, letöltések, értékelések követésére
- Mobilbarát és reszponzív dizájn

Keresés és közösségi funkciók

- Kulcsszavas kereső és szűrő (tantárgy, értékelés alapján)
- Kommentelés, csillagozás
- Legnépszerűbb jegyzetek és új feltöltések kiemelése

Adminisztráció és jogosultságok

- Admin felület a tartalmak, felhasználók és kategóriák kezeléséhez
- Feltöltések jelentése és moderálása
 - Admin eszközök kiterjesztése: Külön felület az egyedi profil CSS kérések kezelésére (jóváhagyás / elutasítás / archiválás), valamint a felhasználók és feltöltések részletes kezelése (törlés, szerkesztés, moderation logs).

Profil egyedi CSS kérések

Bevezetésre került a felhasználói CSS kérelmek kezelése, amely a profil felületén megadható, de csak admin jóváhagyása után érvényesül. Az előnézet a felhasználói élményt javítja és nem ment a szerverre automatikusan.

Admin jóváhagyási UI

Az admin panelben külön táblában listázhatók és jóváhagyhatók a CSS kérések; az elfogadott CSS archiválható és az előző állapot is visszaállítható.

Frontend preview & safe background rules

A preview kliens oldali (style tag injektálás), a SAFE_BG_RULE szabály megakadályozza a background képek kellemetlen ismétlődését, és a preview idején a jobboldali panel rejtésre kerül, hogy az előnézet ne rontsa el az oldalt.

Lokalizációs módosítások

A `t()` segédfüggvény és `lang.php` központi megoldással egy adatbázis-alapú fordítási rendszer lett bevezetve. A fordítások hiányzó kulcsait automatikusan lehet seed-elni. Fordítási kulcsok a `profiles` -hoz a 1543-as ID-tól készültek.

SQL dump & seed eszközök

Hozzáadtuk a `clean_translations.py` , `translations_clean.sql` , `replace_translations_in_dump.py` , `repair_translations.sql` eszközöket, amelyek a dump-ok importjának biztonságát növelik (duplikált bejegyzések eltávolítása, ON DUPLICATE használata, táblák import sorrendje). A `repair_translations.sql` script automatikus backup-ot készít, majd törli a duplikátumokat és létrehozza a megfelelő UNIQUE indexet.

Biztonsági javítások

A kód refaktorálásával `db_prepared()` használata javasolt a lekérdezésekhez, `require_once` és `function_exists` védelem bevezetésre került, a `Message()` segédfüggvény javítja az értesítések konzisztenciáját.

1.3. Technológiai Stack

A Jegyzetár fejlesztése során a következő technológiákat és eszközöket használtuk, amelyek mindegyike szabadon elérhető és bármilyen modern számítógépen telepíthető:

Frontend

- **HTML5, CSS3, JavaScript:** Az alapvető webes technológiák a felhasználói felület kialakításához.
- **Bootstrap:** Reszponzív és mobilbarát dizájn gyors fejlesztéséhez.
- **jQuery:** Egyszerű DOM-manipulációk és AJAX-hívások kezelésére.

Backend

- **PHP (8.2+):** A szerveroldali logika és API-k implementálásához.

Adatbázis

- **MySQL:** Relációs adatbázis-kezelő a felhasználói adatok, fájlok és egyéb információk tárolására.
- **phpMyAdmin:** Az adatbázis adminisztrációjához.

Verziókezelés

- **Git:** Verziókövetés és csapatmunka támogatása.
- **GitHub:** Távoli repository a kód tárolására és megosztására.

Egyéb eszközök

- **XAMPP:** Lokális fejlesztői környezet (Apache, MySQL, PHP) - ingyenesen letölthető bármely operációs rendszerre.
- **Visual Studio Code:** Kódszerkesztő a fejlesztéshez - ingyenes és cross-platform.
- **PHPStorm:** Integrált Fejlesztői Környezet a fejlesztéshez - fizetős verzió elérhető bármeély operációs rendszerre (opcionális).

Felhasznált hardverek

Baranyi Norbert

-

Csontos Kincső Anasztázia

- **Laptop:** Lenovo LOQ 15A

- **CPU:** AMD Ryzen 5 7235HS
- **GPU:** NVIDIA GeForce RTX 3050 6GB Laptop GPU
- **RAM:** 16GB DDR5 4800 MHz
- **SSD:** 512GB NVMe SSD
- **OS:** Windows 11 Pro 64-bit

Szekeres Levente

-

Hardver követelmények

A fejlesztési környezet bármilyen modern számítógépen futtatható. Minimális követelmények:

- **Operációs rendszer:** Windows 10/11, macOS 10.14+ vagy Linux (Ubuntu/Debian)
- **Processzor:** Modern többmagos CPU (pl. Intel i5, AMD Ryzen 5 vagy hasonló)
- **Memória:** Legalább 8GB RAM (ajánlott 16GB a jobb teljesítmény érdekében)
- **Tárhely:** Legalább 50GB szabad hely SSD-n a projekt fájlok és virtuális környezet számára
- **Grafikus kártya:** Integrált vagy dedikált GPU (nem kritikus, mivel webfejlesztésről van szó)

1.4. Fogalomtár (Glossary)

- **2FA:** Kétlépcsős hitelesítés, belépéskor e-mailben kapott kóddal.
- **OAuth:** Külső szolgáltatóval (pl. Discord) történő bejelentkezés.
- **Tag:** Kulcsszó címke a jegyzetek gyorsabb kereséséhez.
- **Moderáció:** Admin felügyelet (jelentések kezelése, tartalom törlés/szerkesztés).
- **Seed:** Kezdő (teszt) adatok betöltése adatbázisba.
- **Dump:** Adatbázis kimentés (.sql) importálható formában.

2. Rendszerarchitektúra

2.1. Magas Szintű Architektúra

A Jegyzetár egy háromrétegű architektúrát követ:

1. **Prezentációs réteg:** A felhasználói felület, amely a frontend technológiákra épül.
2. **Alkalmazásréteg:** A backend logika, amely PHP segítségével valósul meg.
3. **Adatbázis réteg:** A MySQL adatbázis, amely az összes adatot tárolja.

2.2. Komponensek

- **Frontend:** A felhasználói interakciók kezelése és az adatok megjelenítése.
- **Backend:** A logika és az adatbázis műveletek végrehajtása.
- **Adatbázis:** A felhasználói adatok, fájlok és metaadatok tárolása.

2.3. Adatbázis Séma

Az alábbi ábra a Jegyzetár adatbázis fő tábláit és azok kapcsolatait mutatja be:

[Hamarosan]

Jogosultsági szintek (áttekintés)

- **Guest (vendég):** böngészés, keresés, nyilvános jegyzetek megtekintése/letöltése (ha engedélyezett).
- **User (felhasználó):** feltöltés, komment, értékelés, kedvencek, profil szerkesztés, barátok/üzenetek.
- **Admin:** moderáció (jelentések), felhasználók kezelése, tartalmak törlése/szerkesztése, CSS kérelmek jóváhagyása/elutasítása.

3. Backend Architektúra

3.1. Backend funkciók

A Jegyzetár backendje PHP alapon biztosítja az alkalmazás üzleti logikáját, a jogosultságkezelést, az adatbázis-műveleteket, valamint a fájlkezelést és az e-mail alapú folyamatokat.

3.1.1. Felhasználókezelés és autentikáció

- **Regisztráció és belépés** (`reglog.php`)
- **E-mail aktiváció:** regisztráció után aktiváló link küldése és validálása
(*mail-regver.php, reg-ver.php, `tokens` tábla*)
- **Kétlépcsős hitelesítés (2FA):** belépés után e-mail kód küldése és ellenőrzése
(*mail-2fa.php, 2fa.php, `2fa_codes` tábla*)
- **Vendég mód:** a főoldal böngészése bejelentkezés nélkül (korlátozott jogosultság)

3.1.2. Jogosultságok és szerepkörök

- Guest / User / Admin jogosultsági szintek alkalmazása
- Admin funkciók elérése és védelme (moderáció, felhasználókezelés, CSS kérések)

3.1.3. Jegyzetek kezelése (CRUD + metaadat)

- Jegyzetek listázása és részletek megjelenítése (`index.php` , `note.php`)
- Metaadatok kezelése (név, leírás, tantárgy, tag-ek)
- Letöltések kiszolgálása és hozzáférés-ellenőrzés (`download.php`)

3.1.4. Fájlfeltöltés és validáció

- Fájl feltöltés kezelése (`upload.php`)
- Kiterjesztés / méret / (opcionálisan MIME) ellenőrzés
- Fájlnevkezelés és biztonsági védelem (pl. path traversal megelőzés)
- Feltöltött fájlok és metaadatok mentése adatbázisba

3.1.5. Közösségi funkciók

- **Kommentek** kezelése (`comments` tábla)
- **Értékelés** kezelése (`ratings` tábla, egyszeri értékelés logika)
- **Kedvencek**: mentés és lista (`favorites.php` , `favorites` tábla)

3.1.6. Jelentés és moderáció (report rendszer)

- Jelentés beküldése és tárolása (`reports` tábla, `report.php`)
- Admin oldali kezelés: státuszok (open/dismissed/resolved), kezelő rögzítése
- Moderációs műveletek: tartalom/tevékenység kezelése (törlés/szerkesztés, ha implementálva)

3.1.7. Profil és testreszabás (CSS kérelmek)

- Profiladatok kezelése (bio, profilkép, téma)
- **Egyedi CSS kérelem** tárolása és státuszkezelése (`user_custom_css_requests`)
- Jóváhagyás után archiválás (`user_custom_css_archive`) és admin döntési folyamat

3.1.8. Üzenetek, barátok, értesítések

- Barátkérélmek és státuszok kezelése (`friends`)
- Privát üzenetek kezelése (`messages`)
- Rendszerértesítések (`notifys`)

3.1.9. Csoport funkciók

- Csoport létrehozás és kezelés (`groups` , `group_members`)
- Csoporton belüli fájlok: feltöltés, jóváhagyás, moderáció (`group_files`)
- Csoportok integrációja keresésbe / navigációba / értesítésekbe (ha így van megoldva)

3.1.10. Lokalizáció (i18n)

- Nyelvek kezelése (`languages`)
- Fordítások adatbázisban (`translations`)
- `t()` / `lang.php` alapú fordítás betöltés és missing-key seeding támogatás

3.1.11. Adatbázis hozzáférési segédek és biztonság

- Egységes DB hozzáférés wrapper-ek: `db_prepared` , `db_stmt` , `db_query`
- Prepared statement alapú lekérdezések preferálása (SQL injection kockázat csökkentése)
- Include/duplikáció védelem (`require_once` , `function_exists`)
- Egységes felhasználói üzenetek: `Message()` helper

3.2. Adatbázis Kapcsolat

A PHP mysql-t használjuk az adatbázis műveletek végrehajtására.

- Security & prepared statements: A kód nagy részét átdolgoztuk, hogy a `db_prepared($conn, $sql, $types, $params)` wrapper-t használjuk, amely a mysqli prepared statements használatát biztosítja. Ezzel jelentősen csökkent a kockázata az SQL injekcióknak, és egységesebbé vált a lekérdezések kezelése.

3.3. Fájlkezelés

A feltöltött fájlokat a szerveren tároljuk, és a fájlokhoz tartozó metaadatokat az adatbázisban rögzítjük.

4. Fejlesztői Dokumentáció

4.1. Fejlesztői Környezet Beállítása

Quickstart (5 perc)

1. Repo klónozás (XAMPP/htdocs alá)
2. DB import: `assets/sql/jegyzetar.sql` (vagy `jegyzetar_clean.sql`)
3. `assets/php/db.php` beállítása
4. Böngésző: `http://localhost/jegyzetar.eu-src/src/`

Gyakori hibák

- 500 error: hiányzó PHP extension / rossz include path
- DB connection error: rossz host/user/pass/dbname
- Duplikált fordítások: `repair_translations.sql` futtatása vagy clean dump használata

1. Klónozd le a projekt fájljait (pl. ``git clone https://github.com/doomhyena/jegyzetar.eu-src.git``) a helyi szerver gyökérkönyvtárába (pl. ``c:/xampp/htdocs/jeg`
2. Importáld az adatbázist:
 - Nyisd meg a phpMyAdmin-t.
 - Importáld a ``Jegyzetár.sql`` fájlt az ``assets/sql/`` mappából.
3. Konfiguráld az adatbázis kapcsolatot:
 - Nyisd meg a ``db.php`` fájlt.
 - Győződj meg róla, hogy az adatbázis hitelesítési adatok megfelelnek a helyi szerver beállításainak.
4. Indítsd el a helyi szerveret, és navigálj a ``http://localhost/jegyzetar.eu-src/src/`` címre a böngésződben.

4.2. Verziókezelési Stratégia

A Jegyzetár projekt verziókezelése a **Semantikus Verziózás (Semantic Versioning – SemVer)** elveit követi, figyelembe véve, hogy a projekt jelenleg **fejlesztési / béta állapotban** van.

4.2.1. Verziószám felépítése

A verziószám formátuma:

`major.minor.patch`

Példa: `1.0.3`

Rész	Jelentés
major	Nagy, kompatibilitást törő változás
minor	Új funkciók hozzáadása (visszafelé kompatibilis)
patch	Hibajavítások, kisebb módosítások

4.2.2. Fejlesztési (beta) állapot jelölése

A projekt jelenleg **nem tekinthető végleges, éles kiadásnak**, ezért a verziószám **előzetes (beta) jelölést** használ.

Használt forma:

```
[1.X.X]
```

Ez a jelölés azt fejezi ki, hogy:

- a **major verzió (1)** már tervezetten rögzített,
- a **minor és patch értékek még változhatnak**,
- a rendszer funkciói aktív fejlesztés alatt állnak.

Megjegyzés:
A [1.X.X] forma **nem végleges kiadást jelöl**, hanem egy **folyamatosan fejlődő verziócsaládot**.
Ez különösen hasznos iskolai / projektmunka környezetben, ahol a végleges release később történik.

8.2.3. Átmenet végleges verzióra

A projekt akkor tekinthető **első hivatalos kiadásnak**, amikor:

- a fő funkciók stabilak,
- a dokumentáció teljes,
- nincs kritikus hiba.

Ekkor a verziószám például:

```
1.0.0
```

Ezt követően:

- 1.1.0 = új funkciók
- 1.0.1 = hibajavítás
- 2.0.0 = nagy, visszafelé nem kompatibilis változás

4.2.4. Verziózás a CHANGELOG-ban

A változásnapló (docs/CHANGELOG.md) **minden verzióváltást rögzít**.

Fejlesztési állapotban:

```
[1.X.X] - 2026-01-15
Added
• ...
Changed
• ...
Fixed
• ...
```

Végleges verziónál:

```
[1.0.0] - 2026-04-30
Added
• Első stabil kiadás
```

4.2.5. Dátumformátum szabályok

A dokumentációban és a CHANGELOG-ban **egységes dátumformátum** kerül használatra:

```
YYYY-MM-DD
```

Példa:

- 2026-01-15
- 2026-04-30

Ez a formátum:

- egyértelmű,
- nem nyelvfüggő,
- nem keverhető össze (pl. US/EU dátumformátumokkal).

4.2.6. Verziókezelés és Git kapcsolata

- A verziószám **logikai állapotot** jelöl (nem minden commit növeli).
- Verzióváltás akkor történik, ha:
 - nagyobb funkció bekerül,
 - release készül,
 - CHANGELOG frissül.

Git tagek ajánlott formátuma:

```
v1.0.0
v1.1.0
```

4.2.7. Összefoglalás

- A projekt jelenleg **beta / fejlesztési fázisban** van
- A `[1.x.x]` jelölés ezt tudatosan kommunikálja
- A verziózás a **SemVer szabályait követi**
- A végleges kiadás `1.0.0` verzióval történik
- A CHANGELOG és a dokumentáció **összhangban van**

4.3. FájlStruktúra

 A Webalkalmazás fájlstruktúrája

9.4. Fejlesztői eszközök, script-ek és refaktorok

- `assets/php/functions.php` - Központi helyre került a `db_prepared()` segédfüggvény, ami a mysqli prepared statements használatát segíti és csökkenti az SQL injection kockázatát. A fájlban `function_exists` ellenőrzésekkel védjük a többszörös deklarációt.
- Include és duplikációs védelmek: Az include/require helyeken mostantól `require_once` használata ajánlott, és a központi függvényeknél `function_exists` -es feltételek alkalmazása segít elkerülni fatal hibákat többszörös include esetén.
- `assets/php/lang.php` - Feltölti és betölti az adatbázisban tárolt fordításokat (i18n). Egy beépített rutin gondoskodik arról, hogy a hiányzó kulcsok bekerüljenek az adatbázisba a támogatott nyelvekhez, így a `t()` segédfüggvény mindig vissza tud adni fordítást a kulcshoz.
- `assets/js/script.js` - Új kliensoldali funkciók: a profil egyedi CSS előnézete, preview bekapcsolás/letiltás, jobb oszlop rejtése preview közben, és kliens oldali validálás (pl. üres CSS megelőzése).
- `assets/php/functions.php` - A fejlesztés során bevezetett `Message()` segédfüggvény a felhasználói üzenetek központi kiírására szolgál; a korábbi `echo "<script>alert('...')";` szerkezeteket érdemes ezzel kiváltani a konzisztens, könnyebben tesztelhető és lokalizálható felhasználói értesítésekhez.

Fejlesztői script-ek az `assets/sql/scripts/` mappában:

- `clean_translations.py` - Python script, ami a nagy `translations` INSERT tömbből kiszedi a duplikált (`t_key`, `lang_code`) bejegyzéseket, és előállít egy `translations_clean.sql` fájlt, amely csak egyedi fordításokat tartalmaz.
- `replace_translations_in_dump.py` - Segédfájl, amellyel a `jegyzetar.sql` dumptból lecseréljük a `translations` beszúrási blokkot a `translations_clean.sql` tartalmára, és előállítjuk a `jegyzetar_clean.sql` fájlt.
- `repair_translations.sql` - SQL script, amely a már létező adatbázison belül végrehajtható módon:
 1. biztonsági mentést készít a `translations` tábláról (`translations_backup`),
 2. törli a duplikált bejegyzéseket (például megtartja a `MIN(id)` vagy `MAX(id)` bejegyzéseket),
 3. létrehozza a `UNIQUE` indexet `t_key, lang_code` mezőkre.
- Import javaslat: Ha gyakran importálnak db dump-ot, importálják először a séma CREATE TABLE részt, majd az `ALTER TABLE` -okat (kulcsokkal), és végül a seed adatokat (INSERT), vagy használják a `jegyzetar_clean.sql` fájlt amely előre eltávolítja a duplikátumokat.

- A `profile.php` oldalhoz kapcsolódó fordítási kulcsok (HU/EN/DE) seeding-je `ID = 1543` -től készült el. Amennyiben a roll-out során seed fájlokat használunk, ezek az értékek biztosítják a következetes fordítási kulcstartományt.

4.5. Konfiguráció

- DB: `assets/php/db.php`
- Upload path: `src/users/` (jogosultságok!)
- OAuth: `assets/oauth/` + szükséges kulcsok (.env ha van)
- Mail: `assets/php/mail-*.php` (SMTP / sender beállítások)

4.6. Fő folyamatok

Egyedi CSS kérés folyamata

1. User kitölti a CSS mezőt + preview (kliensoldali)
2. Beküldés → `user_custom_css_requests` (pending)
3. Admin panel: approve/reject
4. Approved esetén archiválás → `user_custom_css_archive`

4.6.1. 2fa.php

Oldal neve: `2fa.php`

Cél: A bejelentkezés második lépcsője: az e-mailben kapott egyszer használatos kód ellenőrzése, majd siker esetén beléptetés.

Elérés / route:

- URL: `.../2fa.php`
- Belépés szükséges? **User** (legalább `$_SESSION['id']` és `$_SESSION['email']` megléte kötelező)

Bemenetek (inputok):

- GET: nincs érdemi GET paraméter
- POST:
 - `code` (string): a felhasználó által beírt 2FA kód
- Session:
 - `id` (int): felhasználó azonosító
 - `email` (string): felhasználó email címe
 - `tries` (int): hibás próbálkozások száma (ha nincs, 0-ra inicializálódik)

Folyamat (lépések):

1. Biztonsági headerek beállítása

- `X-Frame-Options: DENY` (clickjacking ellen)

- `X-Content-Type-Options: nosniff`
- `Referrer-Policy: no-referrer`

2. Session előkészítés

- Ha `$_SESSION['tries']` nincs beállítva: `0` értékre állítja.

3. Jogosultság ellenőrzés

- Ha nincs `$_SESSION['id']` vagy `$_SESSION['email']` :

- `redirect` → `reglog.php`
- `exit`

4. Oldal megjelenítés (navbar + űrlap)

- Rendereli az oldalt és a kód bekérésére szolgáló formot.

5. Kód ellenőrzés (POST esetén)

- Ha érkezett `$_POST['code']` :

1. `trim()` → `$code`

2. Ha `$code` nem üres:

- DB lekérdezés: van-e találat a `2fa_codes` táblában a `(userid, code)` párosra:
 - `SELECT id FROM 2fa_codes WHERE userid = ? AND code = ? LIMIT 1`

3. **Siker ág** (pontosan 1 találat):

- Törli a kódot (egyszer használatos):
 - `DELETE FROM 2fa_codes WHERE userid = ? AND code = ?`
- Beállít egy cookie-t:
 - `setcookie("id", $userid, time() + 3600, "/");` (1 óra)
- `session_destroy()` (a 2FA session vége)
- `redirect` → `index.php` + `exit`

4. **Hiba ág** (nincs találat / üres kód):

- JS alert: `Helytelen kód`
- `$_SESSION['tries']++`
- Ha `tries >= 3` :
 - `session_destroy()`
 - `redirect` → `reglog.php` + `exit`

UI állapotok / felhasználói visszajelzés:

- Loading: nincs
- Empty state: nincs külön kezelve
- Error state:
 - Hibás kód esetén: `alert('Helytelen kód')`
 - 3 hibás próbálkozás után: visszadob a `reglog.php` oldalra (session törléssel)
- Success state:
 - Helyes kód esetén: kód törlése + cookie beállítás + redirect `index.php`

Biztonság:

- SQL injection védelem: paraméterezett DB hívások (`db_query` , `db_exec` helyettesítőkkal)
- XSS védelem: itt nincs visszatükrözött user input HTML-be (alert fix szöveg), így közvetlen XSS felület nem látszik
- Brute force védelem: session alapú próbálkozás limit (`tries` , max 3)
- Clickjacking/Content sniffing/Referrer védelem: a fenti HTTP headerek
- CSRF token: **nincs** a formban (jelenleg nem implementált)

Elfogadási kritériumok (tesztelhető):

- "Ha nincs bejelentkezési session (`id` / `email`), az oldal `reglog.php` -ra redirectel."
- "Helyes kód esetén a `2fa_codes` rekord törlődik, `id` cookie beállítódik 1 órára, majd `index.php` redirect történik."
- "Hibás kód esetén `tries` növekszik és a felhasználó figyelmeztetést kap."
- "3 egymást követő hibás kód után a session törlődik és `reglog.php` redirect történik."
- "Üres kódra nem történik DB törlés/'beléptetés', és hibának minősül."

Ha szeretnéd, ugyanebben a sablonban leírom a *2FA kód generálás/kiküldés* oldalát/funkcióját is (ahol a `2fa_codes` rekord létrejön és az email kimegy) - mert a teljes 2FA flow igazán ott válik kerekké.

4.6.2. Admin Panel

Oldal neve: `admin_panel.php`

Cél: Admin felület az oldal tartalmainak és moderációs elemeinek kezelésére: felhasználók/fájlok/kommentek törlése, kategóriák "ürítése", profil CSS kérelmek bírálata, badge-ek kezelése és kiosztása, jelentések kezelése, regisztrációs kódok kezelése.

Elérés / route:

- URL: `.../admin_panel.php`
- Belépés szükséges? **Admin**
 - Cookie alapú azonosítás: `$_COOKIE['id']`
 - Admin jogosultság: `users.admin == 1`

Bemenetek (inputok):

- GET:
 - `delete_type` (string): `user|file|comment|category|user_badge|badge`
 - `delete_id` (int): törlendő rekord azonosítója (több típusnál)
 - `subject` (string): kategória törlésnél a kategória neve (URL-enkódolva érkezhethet)
 - `css_action` (string): `approve|reject`
 - `css_id` (int): CSS kérelem ID
- POST:
 - Badge hozzárendelés:
 - `action=add_user_badge`
 - `user_id` (int), `badge_id` (int)
 - Badge CRUD:
 - `badge_action=create|update`
 - create: `name` , `slug` , opcionálisan `description` , `icon`
 - update: `badge_id` + ugyanazok a mezők
 - Jelentések kezelése:
 - `report_action=resolve|dismiss`
 - `report_id` (int)
 - Regisztrációs kódok:
 - létrehozás: `create_reg_code` + `code` , `description` , `max_uses` , `expires_at`
 - aktiválás/deaktiválás/törlés: `activate_reg_code` / `deactivate_reg_code` / `delete_reg_code` + `reg_code_id`
- Session: a fájl nem sessiont használ auth-hoz, hanem **cookie**-t:
 - `id` cookie kötelező és numerikus

Folyamat (lépések):

1. Biztonsági headerek beállítása

- `X-Frame-Options: DENY` , `X-Content-Type-Options: nosniff` , `Referrer-Policy: no-referrer`

2. Azonosítás cookie alapján

- Ha nincs `$_COOKIE['id']` vagy nem numerikus (`ctype_digit`):
 - `redirect` → `reglog.php` + `exit`
- `userId = (int)$_COOKIE['id']`
- DB: `SELECT * FROM users WHERE id = ? LIMIT 1`
- Ha nincs ilyen user:

- redirect → `reglog.php` + exit

3. Admin jogosultság ellenőrzés

- Ha `users.admin != 1` :
 - HTTP 403 + `exit('Hozzáférés megtagadva...')`
- (A body-ban van egy **második** ellenőrzés is: ott UI-s üzenetet renderel és leáll.)

4. Admin műveletek kezelése (POST/GET)

- **User badge hozzárendelés** (POST `action=add_user_badge`)

1. `user_id` , `badge_id` , `adminId` integerre
2. Ellenőrzi, létezik-e már ilyen sor (`user_badges`):
 - `SELECT id FROM user_badges WHERE user_id=? AND badge_id=? LIMIT 1`
3. Ha nem létezik: `INSERT INTO user_badges (user_id, badge_id, granted_by) VALUES (?, ?, ?)`
4. Kimenet: JS redirect `admin_panel.php` + exit

- **Badge létrehozás / szerkesztés** (POST `badge_action`)

- `create` :
 - validál: `name` és `slug` nem üres
 - `description` / `icon` üresen → `NULL`
 - `INSERT INTO badges (...) VALUES (?, ?, ?, ?)`
- `update` :
 - validál: `badge_id > 0` , `name` és `slug` nem üres
 - `UPDATE badges SET ... WHERE id=? LIMIT 1`
- Kimenet: JS redirect `admin_panel.php` + exit

- **Törlések / kategória kezelés** (GET `delete_type` , `delete_id`)

- `user` :
 - ha nem saját fiók:
 - `DELETE FROM users WHERE id=?`
 - `DELETE FROM files WHERE uploaded_by=?`
 - `DELETE FROM comments WHERE userid=?`
- `file` :
 - `DELETE FROM files WHERE id=?`
 - `DELETE FROM comments WHERE postid=?`
- `comment` :

- `DELETE FROM comments WHERE id=?`
- `category` :
 - `subject` esetén: `UPDATE files SET subject='' WHERE subject=?`
 - (tehát nem a "kategória rekordot" törli, hanem kiüríti a fájlokból)
- `user_badge` : `DELETE FROM user_badges WHERE id=?`
- `badge` : `DELETE FROM badges WHERE id=?`
- Kimenet: JS redirect `admin_panel.php` + exit
- **Profil CSS kérelmek bírálata** (GET `css_action` , `css_id`)
 - `approve` :
 1. lekéri a kérést: `SELECT * FROM user_custom_css_requests WHERE id=? LIMIT 1`
 2. ha megvan:
 - adott sor: `status='approved'` , `reviewed_at=NOW()` , `reviewed_by=adminId`
 - ugyanazon user többi pending kérelme: `status='rejected'` (kivéve a mostani)
 - `reject` :
 - adott sor: `status='rejected'` , `reviewed_at=NOW()` , `reviewed_by=adminId`
 - Kimenet: JS redirect `admin_panel.php` + exit
- **Jelentések kezelése** (POST `report_action` , `report_id`)
 - csak `resolve` vagy `dismiss`
 - DB: `UPDATE reports SET status=?, handled_by=?, handled_at=NOW() WHERE id=?`
 - Kimenet: JS redirect `admin_panel.php#reports` + exit
- **Regisztrációs kód létrehozása** (POST `create_reg_code`)
 1. `code` trim, üres → alert és nem ír DB-t
 2. `max_uses` ha szám → int, különben `NULL`
 3. `expires_at` ha megadva: `datetime-local` formátum `T` cserével + `:00`
 4. DB: `INSERT INTO reg_codes (code, description, max_uses, expires_at, active) VALUES (?, ?, ?, 1)`
 5. Kimenet: alert "Létrehozva" (a page marad)
- **Regisztrációs kód aktiválás/deaktiválás/törlés** (POST)
 - `UPDATE reg_codes SET active=0/1 WHERE id=?`
 - `DELETE FROM reg_codes WHERE id=?`

5. Listák lekérdezése (renderhez)

- `reg_codes` , `users` , `files` , `comments` + `username` join , `categories` DISTINCT `subject` , `css_requests` , `user_badges` joinok, `badge_options` , `user_options` , `badges` , `reports` + cél típus szerinti extra lookup a táblákból (users/groups/files).

6. Kimenet (UI render)

- Táblázatos admin felület szekciókkal:
 - Felhasználók / Fájlok / Kommentek / Kategóriák / CSS kérelmek / User badge kiosztás+lista / Badge CRUD / Jelentések / Regisztrációs kódok

UI állapotok / felhasználói visszajelzés:

- Loading: nincs
- Empty state:
 - Jelentések: "Nincs még jelentés."
 - Reg kódok: "Még nincs regisztrációs kód."
- Error state:
 - Nem admin: 403 "Hozzáférés megtagadva..." (vagy body-ban kártyaüzenet)
 - Reg kód létrehozásnál üres kód: `alert('A kód mező nem lehet üres.')`
- Success state:
 - Több művelet után JS redirect vissza az admin panelre
 - Reg kód létrehozásnál: `alert('Regisztrációs kód létrehozva.')`

Biztonság:

- SQL injection védelem: jellemzően paraméterezett hívások (`db_query` , `db_exec` , `db_stmt`) használata
- XSS védelem: listázásoknál több helyen `htmlspecialchars()` (pl. username, email, subject, CSS, report reason)
- Jogosultság: admin ellenőrzés `users.admin == 1` alapján (cookie → user betöltés → admin flag)
- CSRF token: **nincs** (sem a POST formokban, sem a GET-es "törlés/jóváhagyás" műveleteknél)
- Destruktív műveletek GET-tel: törlés/jóváhagyás részben **GET paraméterekkel** történik, csak `confirm()` JS védelemmel (ez UX, nem biztonság)

Elfogadási kritériumok (tesztelhető):

- "Cookie `id` nélkül vagy nem numerikus `id` -vel a rendszer `reglog.php` -ra redirectel."
- "Nem admin user 403-at kap és nem látja az admin panelt."
- "Saját felhasználó nem törölhető (UI-ban 'Saját fiók', backendben is tiltva van a user delete ágon)."
- "Badge hozzárendelésnél duplikált `user_badges` sor nem jön létre."
- "Jóváhagyott CSS kérelem után a felhasználó többi pending kérelme automatikusan rejected lesz."
- "Regisztrációs kód üres `code` mezővel nem kerül DB-be."
- "Report resolve/dismiss esetén a `reports.status` , `handled_by` , `handled_at` frissül, majd a felület a #reports szekcióra ugrik."

4.6.3. Tanuló csoport létrehozása

Oldal neve: `create_group.php`

Cél: Új tanulócsoporthoz létrehozása. A bejelentkezett felhasználó megadja a csoport nevét/leírását, opcionálisan privátra állítja, majd a rendszer létrehozza a `groups` rekordot és automatikusan belépteti a létrehozót "owner" szerepkörrel.

Elérés / route:

- URL: `.../create_group.php`
- Belépés szükséges? **User** (cookie alapú azonosítás: `$_COOKIE['id']`)

Bemenetek (inputok):

- GET: nincs érdemi GET paraméter
- POST (csak ha `letrehoz` be van küldve):
 - `letrehoz` (submit jelző)
 - `name` (string): csoport neve
 - `description` (string): csoport leírása (opcionális)
 - `is_private` (checkbox): ha be van pipálva → privát csoport
- Cookie:
 - `id` (string/int): bejelentkezett felhasználó azonosítója (csak numerikus fogadható el)

Folyamat (lépések):

1. Biztonsági headerek beállítása

- `X-Frame-Options: DENY`
- `X-Content-Type-Options: nosniff`
- `Referrer-Policy: no-referrer`

2. Jogosultság ellenőrzés

- Ha nincs `$_COOKIE['id']` vagy nem numerikus (`ctype_digit`):
 - `redirect` → `reglog.php`
 - `exit`

3. Input beolvasás és validálás (POST esetén)

- Csak akkor fut, ha `isset($_POST['letrehoz'])`
- `name` és `description` trimelve:
 - `$csoport_nev = trim($_POST['name'] ?? '')`
 - `$csoport_leiras = trim($_POST['description'] ?? '')`
- Privát flag:

- `$privat = isset($_POST['is_private']) ? 1 : 0`

o Tulajdonos:

- `$tulaj_id = (int)$_COOKIE['id']`

o Validálás:

- Ha a csoport neve üres:
 - `alert('A csoport neve kötelező!')`
 - nincs DB írás

4. DB műveletek

o Csoport létrehozása:

- `INSERT INTO groups (name, description, owner_id, is_private) VALUES (?, ?, ?, ?)`

o Ha az insert sikeres (`$inserted > 0`):

1. `$uj_csoport_id = $conn->insert_id`
2. Létrehozó felvétele tagként "owner" role-lal, elfogadott státusszal:
 - `INSERT INTO group_members (group_id, user_id, role, status) VALUES (?, ?, 'owner', 'accepted')`
3. `alert('Csoport sikeresen létrehozva!')`
4. redirect → `group.php?id=<új_id>` + `exit`

o Ha az insert nem sikerül:

- `alert('Hiba történt a csoport létrehozásakor.')`

5. Kimenet (UI)

o Form mezők:

- Csoport neve (input)
- Leírás (textarea)
- Privát csoport checkbox + magyarázó szöveg

o Gombok:

- "Csoport létrehozása" (POST)
- "Mégse, vissza a listához" → `groups.php`

UI állapotok / felhasználói visszajelzés:

- Loading: nincs
- Empty state: nincs
- Error state:

- Üres név: alert "A csoport neve kötelező!"
- DB insert hiba: alert "Hiba történt a csoport létrehozásakor."

- Success state:

- alert "Csoport sikeresen létrehozva!"
- redirect a frissen létrehozott csoport oldalára: `group.php?id=...`

Biztonság:

- SQL injection védelem: paraméterezett DB hívás (`db_exec` placeholder-ekkel)
- XSS védelem: ezen az oldalon a beküldött érték nincs visszarendelve (nincs echo a name/description-ra), így közvetlen XSS felület itt nem látszik
- CSRF token: **nincs** (a POST kérés nincs CSRF-fel védve)
- Auth: cookie `id` ellenőrzés (numerikus), de nem ellenőrzi itt külön, hogy a cookie-ban lévő user valóban létezik-e a users táblában (csak annyit, hogy szám)

Elfogadási kritériumok (tesztelhető):

- "Nem bejelentkezett felhasználó (`id` cookie nélkül / nem numerikus id-vel) a `reglog.php` oldalra kerül."
- "Üres csoportnév esetén nem történik DB írás, és megjelenik a kötelező mező hibaüzenet."
- "Sikeres létrehozáskor létrejön egy rekord a `groups` táblában a helyes `owner_id` és `is_private` értékekkel."
- "Sikeres létrehozáskor létrejön egy rekord a `group_members` táblában `role='owner'` és `status='accepted'` értékekkel a létrehozó userre."
- "Siker esetén pontos redirect történik: `group.php?id=<új_csoport_id> ."`

4.6.4. Kedvencek

Oldal neve: `favorites.php`

Cél: A bejelentkezett felhasználó kedvencnek jelölt jegyzeteinek (fájlok) listázása és megjelenítése, elérés biztosítása a részletekhez és letöltéshez.

Elérés / route:

- URL: `.../favorites.php`
- Belépés szükséges? **User** (cookie alapú azonosítás: `$_COOKIE['id']`)

Bemenetek (inputok):

- GET: nincs érdemi GET input
- POST: nincs (csak megjelenítő oldal)
- Cookie:
 - `id` (string/int): bejelentkezett felhasználó azonosítója (numerikus)
- Session: nincs használva ebben a fájlban

Folyamat (lépések):

1. Jogosultság ellenőrzés

- Ha nincs `$_COOKIE['id']` vagy nem numerikus (`ctype_digit`):
 - redirect → `reglog.php` + `exit`
- Betölti a usert:
 - `SELECT * FROM users WHERE id = ? LIMIT 1`
- Ha nincs találat:
 - redirect → `reglog.php` + `exit`

2. Input validálás

- Nincs klasszikus input validálás (nincs GET/POST), csak cookie `id` típusellenőrzés és user létezés ellenőrzés.

3. DB művelet(ek)

- Értesítések száma (nem közvetlenül a kedvencekhez, de UI-hoz):
 - `SELECT id FROM notifys WHERE toid = ? AND readed = 0`
- Kedvencek lekérése:
 - `SELECT * FROM favorites WHERE user_id = ?`
- Minden kedvenc sorra:
 - `file_id` alapján fájl betöltése:
 - `SELECT * FROM files WHERE id = ? LIMIT 1`
 - ha van találat: hozzáadja a `$favorites` tömbhöz
- Renderelés közben (minden listázott fájlnál):
 - feltöltő username lekérése:
 - `SELECT username FROM users WHERE id = ? LIMIT 1`
 - értékelések átlaga + darabszám:
 - `SELECT IFNULL(AVG(rating),0) as avg, COUNT(*) as c FROM ratings WHERE file_id = ?`

4. Kimenet

- Siker: oldal renderelése a kedvenc fájlok kártyáival
- Üres lista: "Még nincsenek kedvenc jegyzeteid" empty state kártya
- Hiba (auth): redirect `reglog.php`

UI állapotok / felhasználói visszajelzés:

- Loading: nincs

- Empty state:
 - Ha `$favorites` üres: üzenet és ikon ("Még nincsenek kedvenc jegyzeteid")
- Error state:
 - Nincs bejelentkezés / érvénytelen user: redirect `reglog.php`
- Success state:
 - Kedvencek megjelenítése kártyákban:
 - cím (`name`)
 - feltöltő profil link (`profile.php?userid=...`)
 - "Részletek" link: `note.php?id=...`
 - "Letöltés" link: `assets/php/download.php?id=...`

Biztonság:

- SQL injection védelem: paraméterezett lekérdezések (`db_query`) használata
- XSS védelem:
 - Megjelenített dinamikus mezők escape-elve: `htmlspecialchars($f['name'])` , `htmlspecialchars($uploader['username'])` , stb.
- Fájl esetén:
 - A `favorites.php` nem tölt fel fájlt; letöltés külön endpointon (`download.php?id=...`) történik (annak a védelme ott releváns).
- CSRF token: nem releváns (nincs állapotmódosító POST ezen az oldalon)

Elfogadási kritériumok (tesztelhető):

- "Cookie `id` nélkül vagy nem numerikus `id` -vel a felhasználó `reglog.php` -ra redirectelődik."
- "Ha a cookie alapján nem létező user azonosító jön, szintén `reglog.php` -ra redirectelődik."
- "Ha van legalább 1 kedvenc rekord, a hozzá tartozó fájl(ok) kártyái megjelennek."
- "Ha nincs kedvenc, az empty state ('Még nincsenek kedvenc jegyzeteid') jelenik meg."
- "A listában lévő elemeknél a fájlnev és username XSS-védetten jelenik meg (`htmlspecialchars`)."
- "A 'Részletek' link `note.php?id=<file_id>` -re, a 'Letöltés' link `download.php?id=<file_id>` -re mutat."

4.6.5. Elfelejtett jelszó

Oldal neve: `forgotpass.php`

Cél: Elfelejtett jelszó visszaállítása biztonsági kérdés válaszával. 2 lépésben működik: (1) felhasználónév + biztonsági válasz ellenőrzése rate limit-tel, (2) új jelszó megadása és mentése.

Elérés / route:

- URL: `.../forgotpass.php`
- Belépés szükséges? **Guest** (nem kell bejelentkezni)

Bemenetek (inputok):

- GET: nincs
- POST:
 - 1. lépés (biztonsági válasz ellenőrzés):
 - `forg-btn` (submit jelző)
 - `username` (string)
 - `security_answer` (string)
 - `csrf` (string)
 - 2. lépés (új jelszó beállítás):
 - `new-pass-btn` (submit jelző)
 - `password1` (string)
 - `password2` (string)
 - `csrf` (string)
- Session:
 - `csrf` (string): CSRF token
 - `pw_reset_user` (int): melyik user resetelhető
 - `pw_reset_ok` (bool): sikeres 1. lépés jelzője

Folyamat (lépések):

1. Jogosultság ellenőrzés / session előkészítés

- `session_start()`
- CSRF token generálás, ha még nincs: `$_SESSION['csrf'] = bin2hex(random_bytes(32))`
- UI állapot változók: `showSecurityForm`, `showNewPassword`, `success`

2. Input validálás

- Minden POST esetén `check_csrf()` fut:
 - ha token nem egyezik: `exit('CSRF blocked')`
- 1. lépésben: `username` és `security_answer` nem lehet üres → `alert_redirect(error_all_fields_required)`
- 2. lépésben:
 - csak akkor engedett, ha `pw_reset_user` és `pw_reset_ok` be van állítva → különben `exit('Unauthorized')`
 - jelszavak egyezzenek → különben `alert_redirect(passwords_not_match)`
 - min. hossz: 8 → különben `alert_redirect(password_too_short)`

3. DB művelet(ek)

- **Rate limit / lock ellenőrzés** (1. lépés):
 - IP: `$_SERVER['REMOTE_ADDR']`

- `password_reset_attempts` lekérdezés: `attempts` , `locked_until` adott `(username, ip)` párosra
- ha `locked_until` a jövőben van → `alert_redirect(msg_too_many_attempts)`

○ **User + biztonsági válasz ellenőrzés:**

- `SELECT id, security_answer FROM users WHERE username = ? LIMIT 1`

- ha nincs user → `alert_redirect(msg_user_not_found)`

- `password_verify($answer, $user['security_answer'])`

- **Helytelen válasz esetén:**

- ha van rl rekord:
 - `attempts = attempts + 1`
 - ha `attempts >= MAX_ATTEMPTS (5)` :
 - `locked_until = now + 15 perc`
 - `UPDATE password_reset_attempts SET attempts=?, locked_until=?, last_attempt=NOW() ...`
- ha nincs rl rekord:
 - `INSERT INTO password_reset_attempts (username, ip_address, attempts, last_attempt) VALUES (... , 1, NOW())`
- majd: `alert_redirect(msg_wrong_security_answer)`

- **Helyes válasz esetén:**

- törli az rl rekordot: `DELETE FROM password_reset_attempts WHERE username=? AND ip_address=?`
- beállítja sessionben:
 - `pw_reset_user = user.id`
 - `pw_reset_ok = true`
- UI váltás: 2. lépés űrlap megjelenítése

○ **Új jelszó mentése** (2. lépés):

- hash: `password_hash($p1, PASSWORD_DEFAULT)`
- `UPDATE users SET password = ? WHERE id = ? LIMIT 1`
- `session_unset(); session_destroy();`
- `success = true` (siker UI)

4. Kimenet

○ Siker:

- "Jelszó sikeresen megváltoztatva" képernyő + link `reglog.php`

- 1. lépés UI:
 - felhasználónév + biztonsági válasz form (CSRF hidden fielddel)
- 2. lépés UI:
 - új jelszó + megerősítés form (CSRF hidden fielddel)
- Hiba:
 - `alert_redirect(...)` üzenetekkel (a pontos megvalósítás az `alert_redirect` függvénytől függ)

UI állapotok / felhasználói visszajelzés:

- Loading: nincs
- Empty state: nincs
- Error state:
 - Üres mezők → "minden mező kötelező"
 - User nem található
 - Rossz biztonsági válasz
 - Túl sok próbálkozás → lock (15 perc)
 - CSRF blokkolás → "CSRF blocked"
 - Jogosulatlan 2. lépés megnyitás → "Unauthorized"
 - Jelszavak nem egyeznek / túl rövid
- Success state:
 - Siker képernyő + "Ugrás a bejelentkezéshez" (`reglog.php`)

Biztonság:

- SQL injection védelem: paraméterezett DB hívások (`db_query` , `db_exec`)
- XSS védelem:
 - oldálnyelv és CSRF érték escape-elve: `htmlspecialchars($lang)` , `htmlspecialchars($_SESSION['csrf'])`
 - a szövegek fordításból jönnek (`t()`), közvetlen user input nincs visszarendelve
- CSRF védelem: **van**, session token + `hash_equals`
- Rate limit / brute force védelem:
 - `password_reset_attempts` tábla `username + ip` alapon
 - MAX 5 próbálkozás után 15 perces lock (`locked_until`)
- Jelszó tárolás: `password_hash()` (bcrypt/argon a PHP defaulttól függően)
- Megjegyzés: a reset folyamat "biztonsági válasz" alapú, nincs email tokenes megerősítés ebben a flow-ban.

Elfogadási kritériumok (tesztelhető):

- "CSRF token nélkül/hibás tokennel a kérés blokkolódik ('CSRF blocked')."

- "Üres `username` vagy `security_answer` esetén nem indul DB ellenőrzés/jelszó reset."
- "Nem létező user esetén 'user not found' üzenet jön (és nincs jelszó módosítás)."
- "5 rossz biztonsági válasz ugyanarra a username+IP párosra 15 perces lockot eredményez."
- "Helyes biztonsági válasz után megjelenik az új jelszó űrlap, és csak ekkor engedett a jelszócsere."
- "Nem egyező jelszavaknál nem történik DB update."
- "Sikeres jelszóváltáskor frissül a `users.password`, a session törlődik, és a siker képernyő jelenik meg, linkkel a `reglog.php` oldalra."
- "Jogosulatlanul (session jelzők nélkül) a 2. lépés POST nem fut le ('Unauthorized')."

4.6.6. Tanulócsoport

Oldal neve: `group.php`

Cél: Csoport adatlap megjelenítése és kezelése: leírás, privát/nyilvános állapot, tagság (csatlakozás/kilépés), tagok listája, (tulajnak) jelentkezések kezelése, csoport jegyzetek megjelenítése, (tagoknak) feltöltés, (tulajnak) jóváhagyás/elutasítás, valamint csoport törlése. Privát csoportnál a tartalom csak tagoknak/tulajnak látható.

Elérés / route:

- URL: `.../group.php?id=...` (az `id` paramétert a `group_init.php` tipikusan beolvassa)
- Belépés szükséges? **Vegyes**
 - A csoport oldal megnyitható, de **privát csoportnál** a tartalom csak **tagoknak / tulajnak** elérhető.
 - A műveletek (csatlakozás, kilépés, adminisztráció, feltöltés stb.) bejelentkezéshez kötöttek (ez a `group_init.php` / `group_actions.php` feladata).

Bemenetek (inputok):

- GET:
 - `id` (int): csoport azonosító (a betöltéshez; közvetetten a `group_init.php` használja)
- POST (a gombok alapján, tényleges feldolgozás a `group_actions.php`-ban történik):
 - Tagság:
 - `join_group` (csatlakozás)
 - `kilepes` (kilépés)
 - Tag kezelés (tulaj):
 - `remove_member` + `remove_user_id` (tag eltávolítása)
 - `elfogadas` + `kezelt_user_id` (jelentkezés elfogadása)
 - `elutasitas` + `kezelt_user_id` (jelentkezés elutasítása)
 - Csoport jegyzet feltöltés (tag):
 - `uj_jegyzet`
 - `jegyzet_nev` (string)
 - `jegyzet_fajl` (file upload)

- `jegyzet_leiras` (string, opcionális)
- Csoport jegyzet moderálás (tulaj):
 - `jegyzet_elfogadas` + `jegyzet_id`
 - `jegyzet_elutasitas` + `jegyzet_id`
- Csoport törlés (tulaj):
 - `csoport_torles`
- Egyéb (külön endpoint):
 - Csoport jelentése: `assets/php/report.php` felé POST
 - `type=group` , `target_id` , `redirect` , `reason`
- Session/Cookie:
 - Ebben a fájlban közvetlenül nincs cookie/session ellenőrzés, ezt a `group_init.php` és `group_actions.php` végzi (ott derül ki: ki az aktuális user, tag-e, tulaj-e).

Folyamat (lépések):

1. Jogosultság / környezet inicializálás

- Biztonsági headerek beállítása.
- Include-ok:
 - `group_init.php` (csoport + user állapot betöltése: pl. `$csoport_id` , `$csoport_nev` , `$privat` , `$aktualis_felhasznalo_tag` , `$aktualis_felhasznalo_tulaj` , `$aktualis_felhasznalo_pending` , `$tulaj_id` stb.)
 - `group_actions.php` (POST műveletek feldolgozása: csatlakozás/kilépés/elfogadás/elutasítás/feltöltés/jóváhagyás/törlés...)
- Privát csoport gate:
 - ha `privat == 1` ÉS nem tag ÉS nem tulaj:
 - `$hiba_uzenet = "Ez egy privát csoport. A tartalom csak tagoknak látható."`
 - a tartalmi blokkok több helyen `$hiba_uzenet == ""` feltétellel vannak "lezárva"

2. Input validálás

- A megjelenítő fájlban főleg feltételes megjelenítés van.
- A tényleges validálás (pl. csatlakozás, feltöltés, jogosultság) a `group_actions.php` -ban történik.
- A report űrlapnál `redirect` mező `$_SERVER['REQUEST_URI']` -ből kerül kitöltésre és HTML-escape-elve.

3. DB művelet(ek)

- Tagok listája (csak tag/tulaj és nem privát tiltás esetén):

- `SELECT group_members.*, users.username ... WHERE group_id=? AND status='accepted'`

- Függő jelentkezések (csak tulaj):

- `SELECT group_members.*, users.username ... WHERE group_id=? AND status='pending'`

- Csoport jegyzetek (elfogadottak):

- `SELECT group_files.*, users.username ... WHERE group_id=? AND is_approved=1 ORDER BY id DESC`

- Csoport jegyzetek (jóváhagyásra várók, csak tulaj):

- `SELECT group_files.*, users.username ... WHERE group_id=? AND is_approved=0 ORDER BY id DESC`

- A tényleges írárok/törlések/jóváhagyások DB oldalon a `group_actions.php` -ban vannak.

4. Kimenet

- Siker: csoport oldal renderelése, állapottól függő blokkokkal:

- leírás / privát jelzés
- csatlakozás/kilépés gombok
- taglista + (tulajnak) eltávolítás gomb
- (tulajnak) pending kérelmek elfogadás/elutasítás
- jegyzetek listája + (tagoknak) feltöltés
- (tulajnak) várakozó jegyzetek elfogadás/elutasítás
- (tulajnak) csoport törlése

- Hiba:

- privát csoport esetén nem tag: figyelmeztetés szöveg, és a tartalom jelentős része rejtve marad

UI állapotok / felhasználói visszajelzés:

- Loading: nincs

- Empty state:

- nincs leírás: "Ehhez a csoporthoz még nincs leírás megadva."
- nincs tag: "Még nincsenek tagok ebben a csoportban."
- nincs pending: "Nincsenek függőben lévő jelentkezések."
- nincs elfogadott jegyzet: "Még nincsenek elfogadott csoport jegyzetek."
- nincs várakozó jegyzet (tulaj): "Nincs elfogadásra váró jegyzet."

- Error state:

- privát csoport + nem tag: "Ez egy privát csoport. A tartalom csak tagoknak látható."
- jelentés / törlés / elutasítás confirm ablakok (JS confirm)

- Success state:

- a konkrét siker üzenetek/redirectek jellemzően a `group_actions.php` -ban vannak (pl. csatlakozás elfogadása, jegyzet jóváhagyása stb.)

Biztonság:

- SQL injection védelem: paraméterezett lekérdezések (`db_query`)
- XSS védelem:
 - csoport név/leírás, tag nevek, jegyzet adatok escape-elve: `htmlspecialchars()` , leírásnál `nl2br(htmlspecialchars(...))`
 - report redirect mező `ENT_QUOTES` -szal escape-elve
- Fájl esetén:
 - Feltöltés `multipart/form-data` -val történik, de a validálás/mentés nem itt van (várhatóan `group_actions.php` végzi: whitelist/max méret/path traversal védelem ott releváns)
 - Megjelenítésnél a fájl elérési út így készül: `users/<username>/<file_name>` és direkt linkként szerepel (a tényleges biztonság attól függ, a feltöltésnél mennyire "safe" a fájlnev és a mappastruktúra)
- CSRF token: **nincs** a formokon (a POST akciók CSRF nélkül futnak, legalábbis ebben a fájlban)
- Jogosultság:
 - Privát csoport tartalom elrejtés UI-szinten itt történik (`$hiba_uzenet` gate)
 - A tényleges jogosultság-ellenőrzés (pl. ne tudj nem tulajként elfogadni/eltávolítani) a `group_actions.php` felelőssége

Elfogadási kritériumok (tesztelhető):

- "Privát csoportban nem tag felhasználó nem látja a taglistát és a csoport jegyzeteket, és megkapja a privát figyelmeztetést."
- "Tag felhasználó látja az elfogadott taglistát és az elfogadott jegyzeteket."
- "Nem tag és nem pending felhasználó csatlakozás gombot lát; pending állapotban 'függőben' üzenetet lát."
- "Tulajdonos látja a pending jelentkezéseket és tud elfogadni/elutasítani."
- "Tulajdonos tud tagot eltávolítani (kivéve saját magát), és látja a csoport törlése gombot."
- "Tag tud új jegyzetet feltölteni a csoportba (űrlap megjelenik)."
- "Tulajdonos látja az elfogadásra váró jegyzeteket és tudja elfogadni/elutasítani."
- "A megjelenített dinamikus szövegek XSS-védetten jelennek meg (`htmlspecialchars`)."

4.6.7. Tanuló csoportok

Oldal neve: `groups.php`

Cél: A rendszerben elérhető tanuló csoportok listázása és megjelenítése, valamint belépési pont biztosítása új csoport létrehozásához (`create_group.php`) és egy adott csoport megnyitásához (`group.php?id=...`).

Elérés / route:

- URL: `.../groups.php`
- Belépés szükséges? **User** (cookie alapú azonosítás: `$_COOKIE['id']`)

Bemenetek (inputok):

- GET: nincs (a csoport megnyitása linkből megy tovább: `group.php?id=...`)
- POST: nincs
- Cookie:
 - `id` (string/int): bejelentkezett felhasználó azonosító (numerikus ellenőrzéssel)
- Session: nincs ebben a fájlban

Folyamat (lépések):

1. Jogosultság ellenőrzés

- Biztonsági headerek beállítása.
- Ha nincs `$_COOKIE['id']` vagy nem numerikus (`ctype_digit`):
 - `redirect` → `reglog.php`
 - `exit`
- `$aktualis_felhasznalo_id = (int)$_COOKIE['id']` (a lapon érdeemben nem kerül felhasználásra, csak eltárolásra)

2. Input validálás

- Nincs klasszikus input (nincs GET/POST), csak a cookie `id` típusellenőrzése.

3. DB művelet(ek)

- Csoportok lekérése:
 - `SELECT * FROM groups ORDER BY id DESC`
 - *Megjegyzés:* itt közvetlen `$conn->query()` van használva (nincs paraméter, így nem injection-érzékeny ebben a formában).
- Eredmény bejárása és renderelése:
 - mezők: `id` , `name` , `description` , `is_private`

4. Kimenet

- Siker: rendereli a csoportkártyákat:
 - név (link: `group.php?id=...`)
 - privát/nyilvános badge
 - leírás vagy "nincs leírás"
 - "Csoport megnyitása" gomb
- Hiba (auth): `redirect` `reglog.php`
- Üres lista: "Nincs megjeleníthető csoport."

UI állapotok / felhasználói visszajelzés:

- Loading: nincs
- Empty state:
 - Ha nincs egy csoport sem: "Nincs megjeleníthető csoport."
 - Egy adott csoportnál nincs leírás: "Ehhez a csoporthoz még nincs leírás megadva."
- Error state:
 - Nem bejelentkezett user: redirect `reglog.php`
- Success state:
 - Csoportlista megjelenik kártyákban
 - CTA: "Új csoport létrehozása" → `create_group.php`

Biztonság:

- SQL injection védelem:
 - A csoport lista query nem fogad inputot (fix SQL), ezért itt nincs közvetlen injection felület.
 - (Általánosan a projektben paraméterezett hívások javasoltak, de ennél a konkrét querynél nem kritikus.)
- XSS védelem:
 - Csoport név/leírás escape-elve: `htmlspecialchars()` , leírásnál `nl2br(htmlspecialchars(...))`
 - Privát/nyilvános státusz szöveg is escape-elve
- CSRF token: nem releváns (nincs állapotmódosító POST)
- Jogosultság:
 - Belépés cookie alapján kötelező (id numerikus ellenőrzéssel)

Elfogadási kritériumok (tesztelhető):

- "Cookie `id` nélkül vagy nem numerikus `id` -vel a felhasználó `reglog.php` -ra redirectelődik."
- "Ha vannak csoportok a DB-ben, mind megjelenik a listában (id desc sorrendben)."
- "A csoport neve linkként a `group.php?id=<id>` oldalra vezet."
- "Privát csoportnál 'Privát csoport', nyilvánosnál 'Nyilvános csoport' címke látszik."
- "Ha egy csoportnak nincs leírása, a megfelelő empty szöveg jelenik meg."
- "A csoportnév és leírás XSS-védetten jelenik meg (`htmlspecialchars`)."

4.6.8. Főoldal

Oldal neve: `index.php`

Cél: Főoldal: köszöntő + névnap megjelenítése, legfrissebb feltöltések listázása, toplistás ("legjobbba értékelt") jegyzetek megjelenítése. Bejelentkezett felhasználónál kedvencek kezelése (toggle) és értékelés (1–5 csillag) közvetlenül a főoldalról.

Elérés / route:

- URL: `../index.php`
- Belépés szükséges? **Guest is elérheti**, de bizonyos funkciók **User**-hez kötöttek:
 - kedvenchez adás/törlés (POST)
 - értékelés (POST)

Bemenetek (inputok):

- GET: nincs (a lap belső horgonyt használ: `index.php#file-<id>` redirect után)
- POST:
 - Kedvenc toggle:
 - `favorite-btn` (jelző)
 - `favorite_file_id` (int)
 - Értékelés:
 - `rate-btn` (jelző)
 - `rate_file_id` (int)
 - `rating` (int, 1–5)
- Cookie:
 - `id` (string/int): bejelentkezett user azonosítója (numerikus ellenőrzéssel)
- Session: nincs ebben a fájlban

Folyamat (lépések):

1. Jogosultság ellenőrzés / user állapot felmérése

- Biztonsági headerek beállítása.
- `$isLoggedIn = isset($_COOKIE['id']) && ctype_digit($_COOKIE['id'])`
- Ha be van jelentkezve:
 - user lekérés: `SELECT id, username FROM users WHERE id = ? LIMIT 1`
 - értesítések száma: `SELECT id FROM notifys WHERE toid = ? AND readed = 0`
- Meghatározza a megjelenített nevet:
 - bejelentkezve: `username` , különben `t('guest')`

2. Névnap lekérdezés

- `$today = date("m-d")`
- `SELECT nevek FROM namedays WHERE datum = ? LIMIT 1`
- ha nincs találat: `t('nameday_none_today')`

3. Lista lekérdezések (főoldali tartalom)

- "Top rated" (oldalsáv):

- `files` + `ratings` LEFT JOIN
- `AVG(r.rating)` és `COUNT(r.id)`
- rendezés: `avg_rating DESC` , majd `rating_count DESC`
- limit 8

- "Legfrissebb feltöltések":

- `SELECT * FROM files ORDER BY id DESC LIMIT 12`

4. Kedvencek kezelése (POST favorite-btn)

1. Ha nincs bejelentkezve: redirect → `reglog.php`

2. `favorite_file_id` intre castolva

3. Ha `file_id > 0` :

- ellenőrzés: `SELECT id FROM favorites WHERE file_id=? AND user_id=? LIMIT 1`
- ha létezik → `DELETE FROM favorites ...`
- ha nem létezik → `INSERT INTO favorites (file_id, user_id) VALUES (?, ?)`

4. redirect → `index.php#file-<file_id>` + exit

5. Értékelés kezelése (POST rate-btn)

1. Ha nincs bejelentkezve: redirect → `reglog.php`

2. `rate_file_id` , `rating` intre castolva

3. Validálás: `file_id > 0` és `rating` 1...5

4. Ellenőrzés: `SELECT id FROM ratings WHERE file_id=? AND user_id=? LIMIT 1`

- ha van → `UPDATE ratings SET rating=? ...`
- ha nincs → `INSERT INTO ratings (file_id, user_id, rating) VALUES (?, ?, ?)`

5. redirect → `index.php#file-<file_id>` + exit

6. Kimenet (render)

- Hero rész: köszöntés névvel + névnap + "Feltöltés" gomb (bejelentkezve `upload.php` , különben `reglog.php`)

- "Új feltöltések" rács:

- kártyák fájlonként, uploader lekéréssel (`users.username`)
- rating átlag+db kiírás (külön query fájlonként)
- kedvenc gomb:
 - bejelentkezve: POST toggle

- vendég: link `reglog.php`

- értékelés:

- bejelentkezve: radio csillagok (onchange submit), és "Te: X/5"
- vendég: "Értékeléshez jelentkez be."

- Oldalsáv: top rated lista linkekkel `note.php?id=...`

UI állapotok / felhasználói visszajelzés:

- Loading: nincs
- Empty state:
 - kódban nincs külön "nincs találat" UI a friss/top listákra (csak `if ($latest_result && $latest_result->num_rows > 0)` jellegű csendes elrejtés)
- Error state:
 - bejelentkezés nélkül POST művelet → redirect `reglog.php`
 - hibás rating (nem 1–5) vagy `file_id <= 0` : nincs üzenet, csak redirect vissza az anchorra
- Success state:
 - kedvenc toggle és értékelés után: redirect `index.php#file-<id>` (vizuálisan: kedvenc gomb állapota / csillag kijelölés változik)

Biztonság:

- SQL injection védelem:
 - Paraméterezett lekérdezések `db_query` / `db_exec` használatával a user inputot érintő részekben (favorites, ratings, user, nameday, uploader).
 - A "top rated" és "latest" lekérdezések fix SQL-lel mennek (`$conn->query`), input nélkül.
- XSS védelem:
 - Megjelenített dinamikus mezők escape-elve: `htmlspecialchars($displayName)` , fájlnev, uploader név, névnap stb.
- CSRF token: **nincs**
 - A kedvenc/értékelés POST-ok CSRF védelem nélkül futnak.
- Fájl esetén:
 - Itt nincs feltöltés; letöltés külön endpoint (`assets/php/download.php?id=...`)
- Egyéb megjegyzés (teljesítmény):
 - Listázásnál több "N+1" query van (uploader + avg rating + user rating + favorite státusz fájlonként).

Elfogadási kritériumok (tesztelhető):

- "Vendég felhasználó látja a főoldalt, de kedvenchez adás/értékelés POST esetén `reglog.php` -ra redirectelődik."
- "Bejelentkezett usernél a köszöntés a username-et mutatja, és az értesítések száma lekérhető (navbarhoz)."
- "A legfrissebb 12 fájl megjelenik (id szerinti csökkenő sorrendben)."
- "Kedvenc gomb megnyomására a `favorites` táblában a rekord létrejön vagy törlődik (toggle), majd redirect `index.php#file-<id>`."
- "Értékelés 1–5 tartományban mentésre kerül (insert vagy update), majd redirect `index.php#file-<id>`."
- "Hibás rating vagy hibás file_id esetén nem történik DB írás."
- "A megjelenített szövegek XSS-védetten jelennek meg (`htmlspecialchars`)."

4.6.9. Feltöltés

Oldal neve: `upload.php`

Cél:

Az oldal célja, hogy a bejelentkezett felhasználók új jegyzeteket tölthessenek fel a rendszerbe. A feltöltés során a fájl mellett metaadatok (név, leírás, kategória) kerülnek rögzítésre, melyek alapján a jegyzet később kereshető és listázható.

Elérés / route:

- URL: `.../upload.php`
- Belépés szükséges? **User**
Nem bejelentkezett felhasználók automatikusan a bejelentkezési oldalra kerülnek átirányításra.

Bemenetek (inputok):

- GET: nincs
- POST:
 - `name` – a jegyzet megjelenített neve
 - `description` – opcionális szöveges leírás
 - `subject` – kategória / tantárgy
 - `file` – feltöltendő fájl
- Session / Cookie:
 - `id` – bejelentkezett felhasználó azonosítója

Folyamat (lépések):

1. Jogosultság ellenőrzés:
Ellenőrzi, hogy létezik-e érvényes bejelentkezett felhasználó. Ha nem, redirect történik.
2. Input validálás:
Kötelező mezők ellenőrzése, fájl meglétének vizsgálata, fájlnev és méret validálása.
3. DB műveletek:
A fájl elmentése a felhasználó saját könyvtárába, majd rekord beszúrása a `files` táblába `db_prepared()`

használatával.

4. Kimenet:

- siker: redirect a főoldalra vagy a feltöltött jegyzet oldalára
- hiba: JavaScript alert formájában megjelenített hiba

UI állapotok / felhasználói visszajelzés:

- Loading: nincs explicit
- Empty state: nem értelmezett
- Error state: hiányzó mezők, tiltott fájl
- Success state: "Sikeres feltöltés" üzenet + redirect

Biztonság:

- SQL injection védelem: `db_prepared()`
- XSS védelem: megjelenítésnél `htmlspecialchars()`
- Fájlkezelés: whitelistelt kiterjesztések, könyvtár-szintű szeparáció
- CSRF token: nincs (fejlesztési kompromisszum)

Elfogadási kritériumok:

- Hibás vagy hiányos input esetén nem történik adatbázis-írás
- Sikeres feltöltés után pontos redirect történik
- Nem bejelentkezett felhasználó nem érheti el az oldalt

4.6.10. Kereső

Oldal neve: `search.php`

Cél:

A rendszerben elérhető jegyzetek keresése és listázása kulcsszó, rendezési szempont és lapozás alapján.

Elérés / route:

- URL: `.../search.php`
- Belépés szükséges? **Guest**

Bemenetek (inputok):

- GET:
 - `q` – keresőkifejezés
 - `sort` – rendezési mód
 - `page` – lapozás
- POST: nincs
- Session: nincs

Folyamat (lépések):

1. Jogosultság ellenőrzés: nem szükséges.

2. Input validálás:

A keresőkifejezés megtisztítása és hosszellenőrzése.

3. DB művelet:

Paraméterezett SQL lekérdezés a `files` táblára.

4. Kimenet:

- siker: találatok listázása
- hiba: "Nincs találat" állapot

UI állapotok / felhasználói visszajelzés:

- Loading: nincs
- Empty state: nincs találat
- Error state: nincs
- Success state: találati lista

Biztonság:

- SQL injection védelem
- XSS védelem
- CSRF token: nem releváns

Elfogadási kritériumok:

- Keresés nem okoz SQL hibát
- Üres keresés is stabilan működik
- Találatok helyesen jelennek meg

4.6.11. Jegyzet

Oldal neve: `note.php`

Cél:

Egy adott jegyzet részletes megjelenítése, beleértve a letöltést, értékeléseket és kapcsolódó információkat.

Elérés / route:

- URL: `.../note.php?id=`
- Belépés szükséges? **Guest**

Bemenetek (inputok):

- GET:
 - `id` – jegyzet azonosító
- POST:
 - értékelés, kedvencek

- Session / Cookie:
- `id` (opcionális)

Folyamat (lépések):

1. ID validálás (numerikus, létező).
2. Jegyzet adatainak lekérdezése.
3. Kapcsolódó adatok betöltése (feltöltő, rating).
4. Oldal renderelése.

UI állapotok:

- Empty state: jegyzet nem található
- Error state: hibás ID
- Success state: teljes jegyzetoldal

Biztonság:

- SQL injection védelem
- XSS védelem
- Letöltés külön endpointon

Elfogadási kritériumok:

- Nem létező jegyzet nem jelenik meg
- Jogosultság nélküli művelet nem hajtható végre

4.6.12. Jegyzet statisztikák

Oldal neve: `note_stats.php`

Cél:

Egy jegyzet statisztikai adatainak megjelenítése a feltöltő számára.

Elérés / route:

- URL: `.../note_stats.php?id=`
- Belépés szükséges? **User (tulajdonos)**

Bemenetek (inputok):

- GET:
 - `id`
- Session / Cookie:
 - `id`

Folyamat (lépések):

1. Jogosultság ellenőrzés (csak tulajdonos).

2. ID validálás.
3. Aggregált statisztikák lekérdezése.
4. Megjelenítés.

Biztonság:

- SQL injection védelem
- XSS védelem

Elfogadási kritériumok:

- Más felhasználó nem fér hozzá
- Statisztikák helyesek

4.6.13. Profil

Oldal neve: `profile.php`

Cél:

Felhasználói profil nyilvános megjelenítése jegyzetekkel és jelvényekkel.

Elérés / route:

- URL: `.../profile.php?userid=`
- Belépés szükséges? **Guest**

Bemenetek (inputok):

- GET:
 - `userid`
- Session:
 - `id` (opcionális)

Folyamat (lépések):

1. User ID validálás.
2. Profiladatok lekérdezése.
3. Kapcsolódó jegyzetek és badge-ek betöltése.
4. Oldal renderelése.

Biztonság:

- SQL injection védelem
- XSS védelem

Elfogadási kritériumok:

- Nem létező profil nem jelenik meg
- Adatok helyesen jelennek meg

4.6.14. Üzenetek

Oldal neve: `messages.php`

Cél:

Privát üzenetküldés és beszélgetések kezelése felhasználók között.

Elérés / route:

- URL: `.../messages.php`
- Belépés szükséges? **User**

Bemenetek (inputok):

- GET:
 - `with`
- POST:
 - `message`
- Session / Cookie:
 - `id`

Folyamat (lépések):

1. Jogosultság ellenőrzés.
2. Üzenet validálás.
3. Mentés adatbázisba.
4. Beszélgetés frissítése.

Biztonság:

- SQL injection védelem
- XSS védelem

Elfogadási kritériumok:

- Nem bejelentkezett user nem írhat
- Üzenet mentésre kerül

4.6.15. Értesítések

Oldal neve: `notify.php`

Cél:

Felhasználói értesítések megjelenítése és olvasott státusz kezelése.

Elérés / route:

- URL: `.../notify.php`
- Belépés szükséges? **User**

Bemenetek (inputok):

- GET:
 - read
- Session / Cookie:
 - id

Folyamat (lépések):

1. Jogosultság ellenőrzés.
2. Értesítések lekérdezése.
3. Státusz frissítése.
4. Lista megjelenítése.

4.6.16. Bejelentkezés és Regisztráció

Oldal neve: reglog.php

Cél: Regisztráció és bejelentkezés kezelése.

Elérés / route:

- URL: .../reglog.php
- Belépés szükséges? **Guest**

Bemenetek (inputok):

- POST:
 - username
 - password
 - regisztrációs adatok
- Session: több lépcsős auth

Folyamat (lépések):

1. Input validálás.
2. Felhasználó ellenőrzés / létrehozás.
3. Session indítás.
4. Redirect főoldalra.

Biztonság:

- SQL injection védelem
- XSS védelem
- Jelszó hash

Elfogadási kritériumok:

- "Hibás jelszó nem enged be"
- "Sikeres login redirectel"

4.6.17. Email-s visszaigazolás

Oldal neve: `reg-ver.php`

Cél: Regisztrációs e-mail vagy kód megerősítése.

Elérés / route:

- URL: `.../reg-ver.php`
- Belépés szükséges? **Guest**

Bemenetek (inputok):

- GET:
 - `code`
- Session: nincs

Folyamat (lépések):

1. Kód validálás.
2. Felhasználó aktiválása.
3. Redirect loginra.

Biztonság:

- SQL injection védelem
- XSS védelem

Elfogadási kritériumok:

- "Hibás kód nem aktivál"
- "Sikeres aktiválás után login"

4.6.18. Adatvédelmi tájékoztató

Oldal neve: `privacy.php`

Cél: Adatvédelmi tájékoztató megjelenítése.

Elérés / route:

- URL: `.../privacy.php`
- Belépés szükséges? **Guest**

Bemenetek (inputok):

- GET: nincs
- POST: nincs
- Session: nincs

Folyamat (lépések):

1. Statikus tartalom betöltése.
2. Oldal renderelése.

UI állapotok:

- Empty state: nincs
- Success state: tartalom megjelenik

Biztonság:

- XSS védelem (statikus tartalom)

Elfogadási kritériumok:

- "Az oldal bárki számára elérhető"
- "A tartalom helyesen jelenik meg"

4.7 Security checklist (dev)

Security checklist

- Minden DB lekérdezés: `db_prepared()`
- Output escaping: HTML-ben `htmlspecialchars()`
- File upload:
 - MIME ellenőrzés
 - whitelist kiterjesztés
 - max size limit
 - fájlnev randomizálás / path traversal védelem
- Session:
 - session fixation elleni védelem (login után `session_regenerate_id`)
- CSRF (ha nincs): formokhoz token javasolt

4.8 Debug / logging

Logging / debug

- PHP error log: XAMPP Apache/PHP log
- App log: `assets/logs/` (ha használjátok)
- Tipikus debug lépések:
 - `display_errors` devben (productionben OFF)
 - SQL hibák: `mysqli_error` / wrapper logolás

4.9. Változásnapló (CHANGELOG)

A projekt hivatalos változásnaplója a `docs/CHANGELOG.md` fájlban található.
Minden érdemi módosítást itt kell rögzíteni, hogy követhető legyen a fejlesztés és a verziók közötti különbség.

Szabályok

- **A legújabb verzió mindig felül legyen.**
- **Minden módosítást rögzíteni kell** (feature, bugfix, refaktor, eltávolítás, security patch).
- A bejegyzéseket az alábbi kategóriák egyikébe kell sorolni:
 - **Added** – új funkció
 - **Changed** – módosítás meglévő funkción
 - **Fixed** – hibajavítás
 - **Removed** – eltávolított funkció/elem
 - **Security** – biztonsági frissítés
- **Dátum formátum:** `YYYY-MM-DD`
- **Verziózás:** `major.minor.patch` (pl. `1.0.3`)
- **Szerző megjelölése kötelező** a bejegyzés végén (`Changelog by: Név`)

Mikor frissítjük?

- **Minden merge / release után** frissíteni kell.
- Ha több kisebb commitból áll egy változás, akkor a changelogba elég a **összefoglaló** jellegű leírás.

Formátum

```
[1.X.X] - 2025-0X-0X
Added
• ...
Changed
• ...
Fixed
• ...
Removed
• ...
Security
• ...
Changelog by: Neved
```

Példa bejegyzés (mintaként)

```
[1.2.0] - 2026-01-14
Added
• Admin felületen külön lista és jóváhagyási folyamat az egyedi profil CSS kérelmekhez.
Changed
• Fordításkezelés átalakítva adatbázis-alapú `t()` rendszerre, automatikus missing-key seedinggel.
Fixed
```

- Fordítás dump import: duplikált `translations` sorok kezelése és biztonságos import sorrend javaslatok. Security
 - Lekérdezések egységesítése `db\_prepared()` wrapperrel az SQL injection kockázat csökkentésére.
- Changelog by: Csontos Kincső Anasztázia

4.10. Dokumentáció karbantartás

A projekt dokumentációja a `docs/dokumentáció.md` fájlban található.

Mikor frissítjük?

- Minden nagyobb funkció (feature) hozzáadása után
- Adatbázis módosítás (új tábla / mező / index) után
- Biztonsági változtatás vagy refaktor után
- Release / merge után a `docs/CHANGELOG.md` frissítése kötelező

Mit kell ilyenkor frissíteni?

- Tartalomjegyzék (ha új fejezet került be)
- Érintett fejezet(ek) leírása (pl. Backend / Frontend / Deployment)
- DB séma (ha változott)
- Changelog bejegyzés (verzió + dátum + kategóriák + szerző)

Minimum elv

Ha nincs idő mindent átírni, legalább:

1. Changelog + 2) érintett fő folyamat rövid leírása kerüljön be.
Megjegyzés: A változásnapló írásának szabályai és formátuma a **8.9. Változásnapló (Changelog)** fejezetben található.

4.11. Oldalak részletes referenciája

Az alábbi alfejezetek a legfontosabb szerveroldali és kliensoldali folyamatokat írják le oldalanként: route, jogosultság, bemenetek/kimenetek, kapcsolódó include fájlok és adatbázis-táblák, valamint rövid fejlesztői jegyzetek és biztonsági megfontolások.

4.11.1. index.php

- Route: `/src/index.php` — főoldal, vendégek és bejelentkezettek számára.
- Jogosultság: Guest allowed; bizonyos műveletek (kedvenc/értékelés) User-only.
- Includes: `assets/php/db.php` , `assets/php/functions.php` , `assets/php/navbar.php` , `assets/php/footer.php` .
- DB táblák: `files` , `users` , `ratings` , `favorites` , `notifys` , `namedays` .
- Fő folyamatok:
 - Lekérdezi a legfrissebb fájlokat (`SELECT * FROM files ORDER BY id DESC LIMIT 12`).

- Kiszámítja a top-rated listát `AVG(rating)` join-nal `ratings` táblából.
 - Kezeli POST műveleteket: kedvenc toggle és értékelés (insert/update into `favorites` / `ratings`).
- Frontend: csillag értékelés és kedvenc toggle az `assets/js/script.js` -ben található kliensoldali kezeléssel (AJAX vagy form POST lehetőséggel).
- Security: minden állapotmódosító POST ellenőrzze a bejelentkezést és használjon prepared statementeket; javasolt CSRF token bevezetése.

4.11.2. reglog.php

- Route: [src/reglog.php](#) — regisztráció és bejelentkezés kezelése.
- Jogosultság: Guest-only for register/login flows; redirectek sikeres műveletek után.
- Includes:
 - `require_once "assets/php/db.php"`
 - `require_once "assets/php/lang.php"`
 - `require_once "assets/php/functions.php"`
 - `include "assets/php/navbar.php"` (render)
 - `include "assets/php/footer.php"` (render)
- Forms / POST mezők:
 - Regisztráció: `lastname` , `firstname` , `username` , `birthdate` , `gender` , `email` , `reg_code` , `password1` , `password2` , `security_question` (hidden), `security_answer` , `reg-btn` .
 - Belépés: `username` , `password` , `login-btn` .
 - Discord prefill: `$_SESSION['discord_prefill']` (username, email) — ha OAuth jön be.
- SQL / DB műveletek (példák a kódban található lekérdezésekből):
 - `SELECT * FROM reg_codes WHERE code = ? AND active = 1 AND (expires_at IS NULL OR expires_at > NOW()) AND (max_uses IS NULL OR used < max_uses) LIMIT 1`
 - `SELECT id FROM users WHERE username = ? LIMIT 1`
 - `SELECT id FROM users WHERE email = ? LIMIT 1`
 - `INSERT INTO users (lastname, firstname, username, birthdate, gender, email, password, security_question, security_answer, registration_date, admin) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, 0)` (prepared statement via `db_stmt`)
 - `UPDATE reg_codes SET used = used + 1, active = CASE WHEN max_uses IS NOT NULL AND used + 1 >= max_uses THEN 0 ELSE active END WHERE id = ?`
 - DB wrapper-ek használata: `db_query` , `db_stmt` (prepared statements)
- Auth / session / cookie / redirect viselkedés:
 - `session_start()` a fájl elején.
 - Regisztrációnál a rendszer beállítja: `$_SESSION['ver_id'] = $newUserId` , `$_SESSION['email'] = $email` és opcionálisan `setcookie("id", $newUserId, time()+3600, "/")` , majd redirect `mail-regver.php` -re.
 - Belépésnél: jelszó ellenőrzés `password_verify` ; ha `email_verified = 0` → alert; ha `twofa_enabled = 1` → `$_SESSION['id']` + redirect `mail-2fa.php` ; különben tartós cookie (`setcookie(..., expires => time()+30d, httponly, samesite=Lax)`) és redirect `index.php` .
- Fájltrendszer hatás:

- Sikeres regisztrációnál a kód létrehozza a felhasználó mappáját: `users/<username>` (`mkdir, 0777, recursive`).
- Hibakezelés / visszajelzés:
 - A kód hibákra és érvényességre JS `alert()` -eket használ (pl. regisztrációs kód hiánya, felhasználónév/emailexist, jelszó mismatch).
- Biztonsági megjegyzések:
 - Jelszavak és security answer hashelése `password_hash()` -szal.
 - Prepared statements (`db_stmt` / `db_query`) használata SQL injection csökkentésére.
 - Nincs CSRF token a reg/login formoknál jelenleg.
 - Javasolt: `session_regenerate_id()` sikeres belépés után, CSRF tokenek bevezetése, és a JS-alert helyett `Message()` helper használata a konzisztens UI-hoz.
 - Rate limiting szükséges (különösen reg-kód ellenőrzésnél és belépési próbálkozásoknál).

Rövid kivonat (fejlesztői referenciaként): beolvassa a `reg_code` -ot, ellenőrzi a kódot és a felhasználó egyediségét, beszúrja az új `users` rekordot prepared statementtel, növeli a reg-code használataszámát, létrehozza a fizikai user könyvtárat, majd e-mailés verifikációhoz redirectel. Belépésnél ellenőrzi a jelszót, kezeli a 2FA átmenetet, és cookie/session beállításokkal véglegesíti a bejelentkezést.

4.11.3. upload.php

- Route: [src/upload.php](#) — fájl feltöltési végpont (User only).
- Jogosultság: bejelentkezett User szükséges (cookie `id` ellenőrzés: `isset($_COOKIE['id']) && ctype_digit($_COOKIE['id'])`).
- Includes / Header-ek:
 - `require_once "assets/php/db.php"`
 - `require_once "assets/php/lang.php"`
 - `require_once "assets/php/functions.php"`
 - `include 'assets/php/navbar.php'` , `include 'assets/php/footer.php'` (render)
 - Biztonsági headerek: `X-Frame-Options: DENY` , `X-Content-Type-Options: nosniff` , `Referrer-Policy: no-referrer` .
- Forms / POST & FILE mezők:
 - File input: `$_FILES['upload-file']` (required)
 - POST: `name` (display name), `description` , `subject` , `applied_tags` (readonly textarea filled via `kereso_tag.php`), `level` (if `edu_stage` / `edu_level` oszlopok léteznek), `is_public` checkbox (ha `is_public` oszlop van), submit button `upload-btn` .
- SQL / DB műveletek (kódból):
 - Per-user kvóta lekérdezés: `SELECT COALESCE(SUM(file_size), 0) AS used_bytes FROM files WHERE uploaded_by = ?`
 - Profanity ellenőrzés: `SELECT * FROM profanity_filter` és kliens-input ellenőrzése `stripos()` -szal; találat esetén redirect `upload.php?profanity=1` .
 - Beszúrás dinamikus oszlopokkal (példa generált SQL):
`INSERT INTO files (uploaded_by, name, file_name, description, file_path, subject, tags, file_size[, is_public][, edu_stage, edu_level]) VALUES (?, ?, ?, ?, ?, ?, ?, ?[, ?, ?, ?])`

— a kód dinamikusan építi a `colsIns`, `vals`, `types` tömböket, majd `db_stmt($conn, $sql, $types, $vals)` -szal futtatja.

- File handling & validation:
 - Méretkorlát: `MAX_FILE_SIZE = 60 * 1024 * 1024` (60 MB) és felhasználói kvóta `MAX_USER_TOTAL = 60 * 1024 * 1024` (60 MB) szerepel a kódban.
 - Engedélyezett kiterjesztések: `['pdf', 'mp4', 'docx']` és MIME-típusok: `application/pdf`, `video/mp4`, `application/vnd.openxmlformats-officedocument.wordprocessingml.document` (ellenőrzés: `mime_content_type($tmp_name)` és `pathinfo` alapján).
 - Per-user összméret ellenőrzés: ha `used_bytes + file_size > MAX_USER_TOTAL` → hiba üzenet.
 - Profanity: a `profanity_filter` tábla sorait ellenőrzi a `name` és `description` mezőkben; egyezés esetén redirect és alert.
 - Fájlnév ütközés kezelése: ha `file_exists($targetPath)` → új fájlnév `basename_timestamp.ext` képezése.
 - Fizikai mentés: `move_uploaded_file($tmp_name, $targetPath)`; siker esetén INSERT a `files` táblába, majd redirect `upload.php?ok=1`.
 - Ha a `files` tábla tartalmaz `edu_stage` / `edu_level` vagy `is_public` oszlopokat, azok feltöltése is megtörténik (a kód először `SHOW COLUMNS FROM files` -szal ellenőrzi ezeket).
- UI / render:
 - Hibaüzenetek és siker üzenet a query string (`?ok=1`) és `$uploadError` változó alapján jelennek meg toast komponensként.
 - `kereso_tag.php` include kitölti a címkék inputot kliens oldali segítséggel.
- Hibakezelés és visszajelzés:
 - Hibák esetén a `$uploadError` változóba kerül a szöveg, amely `htmlspecialchars()` -szal jelenik meg a UI-ban.
 - Profanity esetén redirect `upload.php?profanity=1` és JS alert.
- Biztonsági megfontolások:
 - SQL injection: a kód `db_query` / `db_stmt` wrapper-eket használ prepared statements-szel.
 - MIME + extension ellenőrzés megtörténik, de további szerveroldali MIME ellenőrzés és vírusellenőrzés (clamav vagy hasonló) javasolt.
 - Fájlnévek normalizálása és ütközés-kezelés: ha ütközés, időbélyegzőt fűz hozzá a fájlnévhez.
 - Path traversal elleni védelem: célmappa statikus `users/<username>/` alatti mentés; továbbá a `file_path` mezőben abszolút pálya kerül tárolásra (érdeemes relatív útvonalat és storage root konstans használni).
 - Kvóta ellenőrzés minimalizálja a felhasználónkénti túlhasználatot.
 - Nincs CSRF token a feltöltő űrlapon (jelenleg hiányzik) — javasolt bevezetni.
 - Javasolt még: fájl-típus szűrés szerver oldali whitelist-tel, tartalom szkennelés, async feldolgozás nagy fájlokhoz.

Rövid kivonat (fejlesztői referenciaként): a `upload.php` cookie-alapú auth-ot végez, ellenőrzi a profanity filtert, fájl- és kvóta validációt hajt végre, fizikailag a `users/<username>/` mappába mozgatja a feltöltött fájlt, majd dinamikusan állítja össze az `INSERT INTO files(...)` prepared statement-et és menti a rekordot; végül `upload.php?ok=1` -re redirectel.

4.11.4. note.php

- Route: [src/note.php](#) — egy jegyzet részletei (`?id=`) (Guest allowed).
- Jogosultság: Guest allowed; állapotmódosító műveletek (kedvenc, komment, értékelés) csak bejelentkezett User-eknek.
- Includes / Headerek:
 - `require_once "assets/php/db.php"`
 - `require_once "assets/php/lang.php"`
 - `require_once "assets/php/functions.php"`
 - `include 'assets/php/navbar.php'` , `include 'assets/php/footer.php'` (render)
 - Biztonsági headerek: `X-Frame-Options` , `X-Content-Type-Options` , `Referrer-Policy` .
- Query / GET / POST mezők:
 - GET: `id` (jegyzet azonosító, ellenőrizve `ctype_digit`)
 - POST (állapotmódosító):
 - Kedvenc toggle: `favorite-btn` , `favorite_file_id` (int)
 - Komment: `comment-btn` , `post_id` (int), `comment-text` (string)
 - Értékelés: `rate-btn` , `rate_file_id` (int), `rating` (int 1...5)
 - Download: link `assets/php/download.php?id=<file_id>` (külön endpoint)
- SQL / DB műveletek (konkrét példák a fájlból):
 - `SELECT * FROM files WHERE id = ? LIMIT 1` — alapjegyzet betöltés
 - `SELECT * FROM users WHERE id = ? LIMIT 1` — feltöltő adata
 - `SELECT IFNULL(AVG(rating),0) as avg_rating, COUNT(id) as rating_count FROM ratings WHERE file_id = ?` — átlag+darab
 - `SELECT id FROM favorites WHERE file_id = ? AND user_id = ? LIMIT 1` — kedvenc státusz
 - `INSERT INTO favorites (file_id, user_id) VALUES (?, ?)` és `DELETE FROM favorites WHERE file_id = ? AND user_id = ?` — toggle
 - `INSERT INTO comments (userid, postid, text) VALUES (?, ?, ?)` — komment beszúrás
 - `SELECT c.*, u.username FROM comments c JOIN users u ON c.userid = u.id WHERE c.postid = ? ORDER BY c.id DESC` — komment lista
 - `SELECT rating FROM ratings WHERE file_id = ? AND user_id = ? LIMIT 1` és `INSERT INTO ratings ... / UPDATE ratings SET rating = ? ...` — értékelés mentése
 - `INSERT INTO notifys (fromid, toid, notifytype, readed) VALUES (?, ?, ?, 0)` — értesítés a feltöltőnek kommentnél (opcionális)
 - A kód használ `db_query` , `db_exec` , `db_stmt` wrapper-eket (prepared statements)
- Fájl-elérési és preview viselkedés:
 - A megjelenített fájl elérési címe a DB-ből és a feltöltő username-jéből épül:
`users/<username>/<file_name>` (biztonság: relatív útvonal használata javasolt)
 - `.pdf` preview iframe, `.mp4` video tag, `.docx` ajánlott letöltés

- Naplózás / események:
 - Minden megtekintésnél `log_file_event($conn, $note_id, $user_id, 'view', null)` hívás szerepel
 - Kedvenc/komment/rate eseményeknél is hívja `log_file_event` -et
- UI / render:
 - Kártya megjelenítés, kedvenc gomb, letöltés gomb, rating csillagok, komment szekció (lista + űrlap)
 - Dinamikus XSS-ellenőrzés: `htmlspecialchars()` -t használnak a megjelenített user/mezőkön
- Biztonsági megjegyzések:
 - Inputok validálása: `id` típusellenőrzés (`ctype_digit`) és intval cast
 - SQL injection elleni védelem: prepared statements wrapper-ek használata
 - Output escaping: `htmlspecialchars()` minden felhasználói szöveg megjelenítésnél
 - CSRF token: nincs jelenleg — érdemes bevezetni az állapotmódosító POST űrlapokra
 - Ellenőrizni kell az elérési út/letöltés védelmét a `download.php` endpointnál

Rövid kivonat: `note.php` betölti a `files` rekordot az `id` alapján, megjeleníti a preview-t, kezeli a kedvenc toggle-t, kommenteket és értékeléseket (mind prepared statements-szel), és naplózza az eseményeket. Outputokat `htmlspecialchars()` -szal védik XSS ellen.

4.11.5. admin_panel.php

- Route: [src/admin_panel.php](#) — adminisztrációs felület (Admin only).
- Jogosultság: cookie `id` ellenőrzés + `SELECT * FROM users WHERE id = ?` és `users.admin == 1` ellenőrzés; ha nincs admin flag → HTTP 403 + exit.
- Includes / Headerek:
 - `require_once "assets/php/db.php"`
 - `require_once "assets/php/lang.php"`
 - `require_once "assets/php/functions.php"`
 - `include 'assets/php/navbar.php'` , `include 'assets/php/footer.php'`
 - Biztonsági headerek a fájl elején
- Admin műveletek / POST & GET paraméterek (konkrétak a kódból):
 - Badge hozzárendelés (POST `action=add_user_badge`): `user_id` , `badge_id` → ellenőrzés + `INSERT INTO user_badges` (ha még nincs)
 - Badge CRUD (POST `badge_action=create|update`): `name` , `slug` , `description` , `icon` , `badge_id` → `INSERT INTO badges` / `UPDATE badges`
 - Destruktív GET műveletek (jelenleg GET): `?delete_type=<user|file|comment|category|user_badge|badge>&delete_id=<id>`
 - `user` : törli a `users` , `files` , `comments` rekordokat (nem töröl saját admin accountot)
 - `file` : `DELETE FROM files WHERE id = ?` + `DELETE FROM comments WHERE postid = ?`
 - `comment` : `DELETE FROM comments WHERE id = ?`

- `category : UPDATE files SET subject = '' WHERE subject = ?`
 - `user_badge / badge : DELETE FROM user_badges / badges WHERE id = ?`
- CSS kérelem kezelése (GET `css_action=approve|reject&css_id=`):
 - approve: `SELECT * FROM user_custom_css_requests WHERE id = ? → UPDATE user_custom_css_requests SET status='approved', reviewed_at=NOW(), reviewed_by=?` és a többi pending `status='rejected'`
 - reject: `UPDATE user_custom_css_requests SET status='rejected', reviewed_at=NOW(), reviewed_by=?`
- Jelentések kezelése (POST `report_action=resolve|dismiss`, `report_id`): `UPDATE reports SET status=?, handled_by=?, handled_at=NOW() WHERE id = ?`
- Regisztrációs kódok (POST `create_reg_code` + activate/deactivate/delete):
 - `INSERT INTO reg_codes (code, description, max_uses, expires_at, active) VALUES (?, ?, ?, ?, 1)`
 - `UPDATE reg_codes SET active = 0/1 WHERE id = ?`
 - `DELETE FROM reg_codes WHERE id = ?`
- Listing queries for render (examples in code):
 - `SELECT * FROM reg_codes ORDER BY id DESC`
 - `SELECT * FROM users ORDER BY id DESC`
 - `SELECT * FROM files ORDER BY id DESC`
 - `SELECT comments.*, users.username FROM comments LEFT JOIN users ON comments.userid=users.id ORDER BY comments.id DESC`
 - `SELECT DISTINCT subject FROM files WHERE subject != '' ORDER BY subject ASC`
 - `SELECT r.*, u.username, rv.username AS reviewer_name FROM user_custom_css_requests r JOIN users u ON r.user_id = u.id LEFT JOIN users rv ON r.reviewed_by = rv.id ...`
 - `SELECT ub.*, u.username, b.name AS badge_name FROM user_badges ub JOIN users u ON ub.user_id = u.id JOIN badges b ON ub.badge_id = b.id`
 - `SELECT r.*, u.username AS reporter_name FROM reports r LEFT JOIN users u ON u.id = r.reporter_id ORDER BY (r.status = 'open') DESC, r.created_at DESC`
- UI / render:
 - Több táblázatos szakasz: felhasználók, fájlok, kommentek, kategóriák, profil CSS kérések, user_badge-ek, badge CRUD, jelentések, regisztrációs kódok
 - Minden destruktív műveletnél JS `confirm()` hívás van a UI-ban
- Biztonsági megjegyzések:
 - Szigorú admin ellenőrzés a fájl elején, de sok destruktív művelet GET paraméterekkel történik — ez nem ideális
 - Minden változtatás prepared statements-szel (`db_exec` / `db_stmt`) történik a kódban
 - Javasolt: minden destruktív akciót POST-ra átvinni + CSRF védelem bevezetése, audit/log táblázat vezetése a moderációs műveletekről
 - XSS elleni védelem: `htmlspecialchars()` a megjelenítésnél

Rövid kivonat: az `admin_panel.php` admin-only oldal, amely a backend CRUD és moderációs műveleteket végzi; a kód sok helyen használ prepared statementeket, de több destruktív művelet GET-kéréssel történik, érdemes ezeket harden-elni POST+CSRF+audit réteggel.

4.11.6. group.php

- Route: [src/group.php](#) — csoport adatlap (`?id=`), tagság és csoport-internal műveletek.
- Includes / Headerek:
 - `require "assets/php/db.php"`
 - `require "assets/php/lang.php"`
 - `require_once "assets/php/functions.php"`
 - `require "assets/php/group_init.php"` (betölti a csoport és user állapotot)
 - `require "assets/php/group_actions.php"` (feldolgozza a POST akciókat)
 - `include "assets/php/ads.php"` , `include 'assets/php/navbar.php'` , `include 'assets/php/footer.php'`
 - Biztonsági headerek: X-Frame-Options, X-Content-Type-Options, Referrer-Policy
- Változók és inicializáció (a `group_init.php` által beállított állapotok):
 - `$csoport_id` , `$csoport_nev` , `$csoport_leiras` , `$privat` (0/1),
 - `$aktualis_felhasznalo_tag` (bool), `$aktualis_felhasznalo_tulaj` (bool), `$aktualis_felhasznalo_pending` (bool), `$tulaj_id`
- GET / POST mezők / akciók (kód és `group_actions.php` alapján):
 - POST:
 - `join_group` (submit) — csatlakozási kérelem
 - `kilepes` — kilépés a csoportból
 - `remove_member` + `remove_user_id` — tulaj által tag eltávolítása
 - `elfogadas` + `kezelt_user_id` — tulaj elfogadja a pending tagot
 - `elutasitas` + `kezelt_user_id` — elutasítás
 - Csoport jegyzet feltöltése: `uj_jegyzet` , `jegyzet_nev` , `jegyzet_fajl` (file), `jegyzet_leiras`
 - Csoport jegyzet moderálás (tulaj): `jegyzet_elfogadas` + `jegyzet_id` , `jegyzet_elutasitas` + `jegyzet_id`
 - Csoport törlése: `csoport_torles` (tulaj)
 - GET: nincs destruktív GET művelet közvetlenül a fájlban (a `group_actions.php` kezeli a változtatásokat), de a UI használ formokat POST-hoz
- SQL / DB műveletek (példák a fájlból):
 - Tagok listázása: `SELECT group_members.*, users.username FROM group_members, users WHERE group_members.user_id = users.id AND group_members.group_id = ? AND group_members.status = 'accepted'`
 - Függőben lévő tagok: `SELECT ... WHERE group_members.group_id = ? AND group_members.status = 'pending'`

- Csoport fájlok (elfogadott): `SELECT group_files.*, users.username FROM group_files, users WHERE group_files.uploaded_by = users.id AND group_files.group_id = ? AND group_files.is_approved = 1 ORDER BY group_files.id DESC`
 - Várólistás fájlok (tulajnak): `SELECT ... WHERE group_files.group_id = ? AND group_files.is_approved = 0`
 - A fájlokhoz való elérési cím: `users/<username>/<file_name>` (letöltés / megnyitás linkek)
 - Megjegyzés: a tényleges írások (csatlakozás, kilépés, jegyzet feltöltés/jóváhagyás/törlés) a `group_actions.php` -ban történnek (ott vannak a `INSERT` / `UPDATE` / `DELETE` műveletek)
- UI / render:
 - Csoport adatlap (név, leírás, privát/nyilvános badge), tagok lista, pending lista (tulajnak), csoport jegyzetek (elfogadott + pending)
 - Tagként megjelenik feltöltő űrlap (multipart) és a feltöltött jegyzetek listája
 - Tulajdonosnak külön műveleti gombok: elfogadás/elutasítás, tag eltávolítása, csoport törlése
- Biztonsági megjegyzések:
 - Privát gate a `$hiba_uzenet` változóval: ha privát és a user nem tag/tulaj, a tartalom nagy része rejtve marad
 - Fontos, hogy a `group_actions.php` szigorúan ellenőrizze a jogosultságokat minden írási műveletnél (pl. csak tulaj fogadhat el, törölhet fájlokat)
 - Fájlfeltöltés a csoportba: multipart/form-data, ellenőrizni kell file type/size/kvóta és path traversal elleni védelmet
 - Prepared statements használata javasolt a `group_actions.php` -ban (a projekt wrapper-ei elérhetők: `db_query` , `db_exec` , `db_stmt`)

Rövid kivonat: `group.php` a csoport megjelenítésért felel, a csoport állapotát `group_init.php` tölti be, és a változtatásokat (`join` , `leave` , `accept` , `upload` , `approve`) a `group_actions.php` kezeli; privát csoportok tartalmát csak jogosult felhasználók látják.

4.11.7. create_group.php

- Route: `/src/create_group.php` — új csoport létrehozása (User only).
- Jogosultság: bejelentkezett User.
- Includes: `assets/php/db.php` , `assets/php/functions.php` , `assets/php/group_actions.php` (ha új csoport logikát ott kezelik).
- DB táblák: `groups` , `group_members` .
- Fő folyamatok:
 - Űrlap validálás, `INSERT INTO groups` , és az új csoporthoz owner role hozzáadása `group_members` -hez.
- Security: input sanitization; megakadályozni dupla létrehozást és validálni a tulajdonos ID-t.

4.11.8. favorites.php

- Route: `/src/favorites.php` — kedvencek listája (User only).
- Jogosultság: bejelentkezett User.

- Includes: `assets/php/db.php` , `assets/php/functions.php` .
- DB táblák: `favorites` , `files` , `users` .
- Fő folyamatok:
 - Lekéri a `favorites` -t `user_id` alapján, majd a hozzá tartozó `files` rekordokat listázza.
- Security: ellenőrizni kell, hogy a `favorites` lekérdezés csak a kért user adatait adja vissza; XSS escape a megjelenítésnél.

4.11.9. forgotpass.php

- Route: `/src/forgotpass.php` — jelszó visszaállítási flow (Guest).
- Jogosultság: Guest.
- Includes: `assets/php/db.php` , `assets/php/functions.php` .
- DB táblák: `users` , `password_reset_attempts` .
- Fő folyamatok:
 - Kétlépcsős folyamat: biztonsági kérdés/státusz ellenőrzése + új jelszó beállítása; rate-limiting `password_reset_attempts` -ben.
 - CSRF token használata és session flag (`pw_reset_user` , `pw_reset_ok`).
- Security: erős rate limiting, CSRF ellenőrzés, jelszó erősség validáció.

4.11.10. 2fa.php

- Route: `/src/2fa.php` — kétlépcsős hitelesítés ellenőrzése (User in 2FA state).
- Jogosultság: átmeneti; session-ben tárolt `id` és `email` szükséges.
- Includes: `assets/php/db.php` , `assets/php/mail-2fa.php` (kód küldéshez), `assets/php/functions.php` .
- DB táblák: `2fa_codes` .
- Fő folyamatok:
 - POST `code` ellenőrzése a `2fa_codes` tábla ellen; siker esetén törli a kódot és befejezi a beléptetést.
 - Hibák kezelése session `tries` számlálóval és 3 sikertelen próbálkozás után session destroy + redirect.
- Security: kódok egyszer használatosak; limitált élettartam; ne tároljuk tisztán a kódot hosszú távon.

Megjegyzés: ez a szakasz alapvetően a `src/` könyvtárban található fájlok jelenlegi felépítésére épít. Szeretném, hogy automatikusan generáljak belőle egy részletes TOC-be illeszthető mellékletet, vagy folytassam a többi, kisebb oldal (pl. `messages.php` , `notify.php` , `premium.php` , `payment_demo.php` , stb.) részletes leírásával is?

4.11.11. groups.php

- Route: `/src/groups.php` — tanulócsoporthoz tartozók listája és belépési pont.
- Jogosultság: User (cookie-based auth required).
- Includes: `assets/php/db.php` , `assets/php/functions.php` .
- DB táblák: `groups` , `group_members` .
- Fő folyamatok:

- Listázza a `groups` táblát, megjeleníti a csoportkártyákat, CTA: `create_group.php` .
 - Paginate/sort lehetőség (ha implementálva).
- Security: csak a belépett user látja; XSS escape a csoportnévnél/leírásnál.

4.11.12. messages.php

- Route: [src/messages.php](#) — privát üzenetküldés és beszélgetések kezelése.
- Jogosultság: bejelentkezett User szükséges (`isset($_COOKIE['id']) && ctype_digit($_COOKIE['id'])`).
- Includes / Headerek:
 - `require_once "assets/php/db.php"`
 - `require_once "assets/php/lang.php"`
 - `require_once 'assets/php/functions.php'`
 - `include 'assets/php/navbar.php' , include 'assets/php/footer.php'`
 - Biztonsági headerek: `X-Frame-Options` , `X-Content-Type-Options` , `Referrer-Policy` .
- Inputs / POST fields:
 - POST: `send_message` submit, `toid` (int), `message` (string)
 - GET: `friendid` to select conversation partner
- SQL / DB műveletek (konkrét a kódból):
 - Auth user lekérése: `SELECT * FROM users WHERE id = ? LIMIT 1`
 - Friend list: `SELECT * FROM friends WHERE (fromid = ? AND status = 1) OR (toid = ? AND status = 1)` — majd minden friend-hez `SELECT * FROM users WHERE id = ?` a megjelenítéshez
 - Üzenet küldés: `INSERT INTO messages (fromid, toid, content, sent_at) VALUES (?, ?, ?, NOW())` (via `db_exec`)
 - A beszélgetés betöltéséhez a kód include-olja az `assets/php/loadmessages.php` -t (az tartalmazza a konkrét SELECT lekérdezést és render logikát)
- UI / render:
 - Oldal kétszlopos: bal oldalon barátlista, jobb oldalon üzenetablak a kiválasztott `friendid` -hez
 - Üzenetek konténer: `#message-container` betölti a `loadmessages.php` kimenetét; van beviteli mező + `send_message` gomb
 - Kliensoldali frissítés lehet AJAX polling vagy WebSocket (ha `loadmessages.php` támogatja)
- Biztonsági megjegyzések:
 - Input validálás: `toid` és `friendid` integer cast/ellenőrzés, `message` trim és üres ellenőrzés
 - SQL injection: `db_exec` és `db_query` wrapper-ek (prepared statements) használata
 - XSS: üzenetek megjelenítésekor `htmlspecialchars()` javasolt (a `loadmessages.php` -ben kell biztosítani)
 - Rate limiting: nincs expliciten a fájlban, de javasolt a spam elleni védelem (pl. per-user cooldown)

Rövid kivonat: `messages.php` barátokat listáz, betölti a kiválasztott beszélgetést (`loadmessages.php`), és POST `send_message` -re beszúrja az `messages` táblába az üzenetet, majd redirect a `?friendid=` anchorra.

4.11.13. notify.php

- Route: [src/notify.php](#) — felhasználói értesítések listázása és kezelése.
- Jogosultság: bejelentkezett User (`isset($_COOKIE['id']) && ctype_digit($_COOKIE['id'])`).
- Includes / Headerek:
 - `require "assets/php/db.php"`
 - `require "assets/php/lang.php"`
 - `require_once "assets/php/functions.php"`
 - `include 'assets/php/navbar.php' , include 'assets/php/footer.php'`
- POST / Actions (konkrét a forrásból):
 - `group_invite_accept : notif_id , group_id` → ellenőrzi a csoportot, ha nincs tagság, beszúr `group_members (group_id, user_id, role='member', status='accepted')` , majd törli a notif-ot (`DELETE FROM notifys WHERE id = ?`)
 - `group_invite_decline : notif_id` → törli a notif-ot
 - `del-notifs-btn` : törli az összes értesítést a user-hez: `DELETE FROM notifys WHERE toid = ?`
- Listing / Queries:
 - Nem olvasott count: `SELECT * FROM notifys WHERE toid = ? AND readed = 0`
 - Teljes lista: `SELECT * FROM notifys WHERE toid = ? ORDER BY id DESC`
 - A megjelenítésnél a `notifier` (fromid) adatait lekéri: `SELECT * FROM users WHERE id = ? LIMIT 1`
 - Group invite esetén lekérdezi a csoportot: `SELECT * FROM groups WHERE id = ? LIMIT 1`
 - Végén mark-as-read: `UPDATE notifys SET readed = 1 WHERE toid = ?`
- UI / render:
 - Lista kártyákban: típus szerint (`friend` , `comment` , `group_invite`) külön viselkedés és gombok
 - `group_invite` tartalmaz `Meghívás elfogadása` / `Elutasítás` formokat
 - Van `Üzenetek törlése` gomb az összes értesítés törléséhez
- Biztonsági megjegyzések:
 - Ellenőrizni kell, hogy a műveletek csak a `toid` -hoz tartozó értesítésekre hatnak (auth check)
 - Prepared statements használata a bemenet-dependent SQL-eknél
 - XSS: a `reason` mezőt `htmlspecialchars()` -szal jelenítik meg (`nl2br(htmlspecialchars(...))` látható a kódban)

Rövid kivonat: `notify.php` listázza a felhasználó értesítéseit, lehetőséget ad meghívások elfogadására/elutasítására, és mark-as-read/clear all gombokat biztosít; a csoport elfogadás beszúrja a `group_members` rekordot és törli az értesítést.

4.11.14. premium.php

- Route: [src/premium.php](#) — prémium szolgáltatások oldal és demo checkout trigger.
- Jogosultság: bejelentkezett User (`isset($_COOKIE['id'])`) — ha nincs, redirect `reglog.php?mode=login`).
- Includes / Headerek:
 - `require "assets/php/db.php"`
 - `require "assets/php/lang.php"`
 - `require "assets/php/premium.php"` (tartalmazza a `user_premium()` és `premium_datum()` függvényeket)
 - `include "assets/php/navbar.php"` , `include "assets/php/footer.php"`
- Logic / Queries:
 - `$premium_var = user_premium($conn, $uid)` — helper ellenőrizheti a `users` tábla vagy `payments` tábla státuszát
 - `$premium_ig = premium_datum($conn, $uid)` — visszaadott lejárat dátum (ha van)
 - UI jelzi `?paid=1` query string esetén a sikeres (demo) fizetést
- UI / Actions:
 - Ha nincs prémium: lista előnyökről és gomb a demo fizetéshez (`form action="payment_demo.php" method="get"`)
 - Ha van prémium: megjeleníti az aktív dátumot és letiltja a fizetés gombot
- Biztonsági megjegyzések:
 - A fizetési folyamatot (`payment_demo.php` /éles payment endpoint) külön kell védeni és validálni a visszahívásokat (webhook/signature)
 - Ne jelenítsünk meg érzékeny kártyainfót; használd a payment provider tokenizációját
 - `user_premium()` és `premium_datum()` implementációját auditáld (honnan származik a státusz: `users` mező vagy `payments` tábla?)

Rövid kivonat: `premium.php` a prémium státuszt jeleníti meg a felhasználónak (`user_premium` + `premium_datum`), és irányít a demo/checkout oldalra; a tényleges fizetési logika a `payment_demo.php` - ban/alkalmazott payment provider integrációban van.

4.11.15. payment_demo.php

- Route: [src/payment_demo.php](#) — demó/sandbox fizetési végpont (tesztelésre).
- Jogosultság: Guest allowed, de csak fejlesztői környezetben használni (nem éles).
- Includes: `require_once "assets/php/db.php"` , `require_once "assets/php/functions.php"` .
- POST / GET mezők:
 - POST: `pay` (submit), `card_number` , `expiry` , `cvv` , `name_on_card` , `uid` (opcionális felhasználó azonosító teszthez).
 - GET: opcionális `demo=1` jelzés a UI-nak.

- Fő folyamatok (kódból):
 - Validálja a demo kártyaszámot (pl. `4242424242424242`) és egyszerű CVV/expiry ellenőrzést.
 - Ha a demo ellenőrzés sikeres, meghívja a `premium_aktivalas_30nap($conn, $uid)` segédfüggvényt, INSERT/UPDATE a `premium_users` táblában, majd redirect `premium.php?paid=1` .
 - Opcióként létrehoz egy `payments` rekordot sandbox adatokkal (nem tartalmaz valódi kártyaadatot), pl. `INSERT INTO payments (user_id, amount, provider, status, created_at) VALUES (?, ?, 'demo', 'success', NOW())` (kód szerint).
- Fájlrendszer / állapotváltozás: csak DB-be ír sandbox fizetési rekordot és prémium státuszt módosít.
- Biztonsági megjegyzések:
 - Nem szabad éles kártyaadatot gyűjteni ezen az endpointon.
 - A valódi payment provider integrációkhoz webhook signature/verification és tokenizáció szükséges.
 - Audit logolás javasolt minden fizetési státusz-változáshoz.

4.11.16. privacy.php

- Route: [src/privacy.php](#) — adatvédelmi tájékoztató (főleg statikus tartalom).
- Jogosultság: Guest allowed.
- Includes: `require_once "assets/php/db.php"` (opcionális), `require_once "assets/php/lang.php"` , `require_once "assets/php/functions.php"` , `include 'assets/php/navbar.php'` , `include 'assets/php/footer.php'` .
- Fő funkciók:
 - Rendereli és lokalizálva megjeleníti a privacy szabályzatot (`t()` fordítási helper használható a szövegre).
 - Nem hajt végre DB műveleteket (tartalom statikus vagy CMS-ben tárolt lehet).
- Biztonsági megjegyzések:
 - A dokumentumot XSS-től mentesen kell tárolni és megjeleníteni (`htmlspecialchars()` ahol dinamikus tartalom szerepel).
 - Ne tartalmazzon érzékeny belső linkeket vagy debug információt production környezetben.

4.11.17. terms.php

- Route: [src/terms.php](#) — felhasználási feltételek (statikus oldal).
- Jogosultság: Guest allowed.
- Includes: `require_once "assets/php/db.php"` (opcionális), `require_once "assets/php/lang.php"` , `require_once "assets/php/functions.php"` , `include 'assets/php/navbar.php'` , `include 'assets/php/footer.php'` .
- Fő funkciók:
 - Megjeleníti a szolgáltatási feltételeket, jogi szöveget és linkeket a privacy/szabályzatokhoz.
- Biztonsági megjegyzések:
 - A jogi szöveget frissítse a jogi csapat jóváhagyásával; kerülje a dinamikus beágyazott tartalom használatát, kivéve ha auditált.

4.11.18. profile.php

- Route: [src/profile.php](#) — felhasználói profil és szerkesztési felület.
- Jogosultság: Guest can view public profiles; full edit actions require cookie `id` auth and matching user id (owner-only).
- Includes: `require_once "assets/php/db.php"` , `require_once "assets/php/lang.php"` , `require_once "assets/php/functions.php"` , `include 'assets/php/navbar.php'` , `include 'assets/php/footer.php'` .
- GET / POST / FILE mezők:
 - GET: `userid` vagy `username` (profile lookup).
 - POST (edit saját profil): `edit_profile` submit, `lastname` , `firstname` , `display_name` , `bio` , `website` , `custom_css_request` (text), `toggle_2fa` (on/off), `profile_pic` (`$_FILES` upload).
 - File upload: `profile_pic` → saved to `users/<username>/` via `move_uploaded_file()` .
- DB műveletek (konkrét példák a kódból):
 - `SELECT * FROM users WHERE id = ? LIMIT 1` — profil betöltés (prepared `db_query`).
 - Profil update: `UPDATE users SET firstname=?, lastname=?, bio=?, website=? WHERE id = ?` (via `db_stmt`).
 - Custom CSS kérelem: `INSERT INTO user_custom_css_requests (user_id, css, status, created_at) VALUES (?, ?, 'pending', NOW())` .
 - Jelkép/felhasználói fájlok lekérése: `SELECT * FROM files WHERE uploaded_by = ? ORDER BY id DESC` .
- Fájrendszer hatások:
 - Profilkép mentése: `users/<username>/<filename>` ; a feltöltésnél fájlnev-ütközés és MIME-check történik.
 - Saját fájlok listázása a felhasználói mappából a `files` tábla alapján.
- Biztonsági megjegyzések:
 - Szigorúan ellenőrizze, hogy csak a tulajdonos tudja szerkeszteni az adatokat (`$_COOKIE['id']` ellenőrzése és egyezés).
 - Fájl feltöltésnél MIME/extension/size ellenőrzés és `move_uploaded_file()` ; normalizálja a fájlneveket, kerülje a path traversal-t.
 - Approved custom CSS esetén a CSS inline beillesztése potenciális XSS/CSS exfiltration kockázatot jelent — csak admin-approved CSS-t engedélyezzen és sandboxoljon.
 - Használjon `htmlspecialchars()` minden felhasználói szöveg megjelenítésnél.

4.11.19. reg-ver.php

- Route: [src/reg-ver.php](#) — e-mail aktivációs / verifikációs flow.
- Jogosultság: Guest allowed.
- Includes: `require_once "assets/php/db.php"` , `require_once "assets/php/functions.php"` , `require_once "assets/php/mail-regver.php"` .
- GET mezők:
 - `token` vagy `code` (aktiváló token a mailből).
- Fő műveletek (kódból):
 - Ellenőrzi a `$_SESSION['ver_id']` és a GET `token` értékét, keresi a `tokens` táblában a megfelelőt (egyszer használatos, lejáratí idővel).

- Siker esetén: `UPDATE users SET email_verified = 1 WHERE id = ?` (prepared statement), törli vagy invalidálja a `tokens` sort és redirect `reglog.php?verified=1` .
 - Hiba/lejárt token esetén visszajelzés, nem publikusan részletezett hibaüzenet a támadási felület csökkentésére.
- Biztonsági megjegyzések:
 - Tokenek egyszeri használatúak és lejáratl kell rendelkezniük; használjon prepared statements.
 - Ne publikusítsa a belső hibákat; használjon rate limiting-et a verifikációs kéréseknél.

4.11.20. search.php

- Route: [src/search.php](#) — kereső és találati oldal.
- Jogosultság: Guest allowed.
- Includes: `require_once "assets/php/db.php"` , `require_once "assets/php/functions.php"` , `include 'assets/php/kereso_tag.php'` (autocomplete/tags).
- GET / POST mezők:
 - GET: `q` (keresőkifejezés), `scope` (`files|users|all`), `page` (lapozás), `type` , `edu_stage` , `edu_level` (szűrők), `sort` .
- Fő logika (kódból):
 - A `tokenize_query()` helper-rel a `q` lekérdezést tokenizálja és dinamikusan építi a WHERE/score részeket.
 - Keresés lehet fájlokra és felhasználókra; a kód dinamikusan állítja össze a SQL-t és a `types` / `params` tömböt a `db_stmt` híváshoz.
 - Paging: offset alapú és/vagy keyset logika a nagy eredményhalmazokra.
 - Javaslat: “did you mean” egyszerű spellcheck/token-suggest logika a `tokenize_query()` alapján.
 - Találati súlyozás: részleges egyezés, cím/description előnyben részesítése, rating/népszerűség befolyásolása (ha alkalmazott).
- Kimenet / session hatás:
 - A találat-azonosítókat session-ben (`search_ids`) tárolhatja a UI gyors részletekhez.
- Biztonsági megjegyzések:
 - Mindig prepared statements-t használjon a dinamikus SQL paraméterezésnél.
 - Limitálja a `q` hosszát és korlátozza az offset-et (pl. max 1000 találat lapozás).
 - A költséges tokenizáció/párosítás háttérben cache-elhető.

4.11.21. note_stats.php

- Route: [src/note_stats.php](#) — részletes jegyzetstatisztikák (feltöltő/owner only).
- Jogosultság: csak a fájl tulajdonosa vagy admin láthatja (cookie `id` és fájl `uploaded_by` ellenőrzés).
- Includes: `require_once "assets/php/db.php"` , `require_once "assets/php/functions.php"` .
- Főbb lekérdezések / aggregációk (kódból):
 - Összes nézet/download/fav: `SELECT SUM(views) AS total_views, SUM(downloads) AS total_downloads FROM file_stats_daily WHERE file_id = ?` vagy `SELECT COUNT(*) FROM file_events WHERE file_id = ? AND type='view'` .

- Értékelések: `SELECT AVG(rating) as avg_rating, COUNT(id) as rating_count FROM ratings WHERE file_id = ?` .
- Utolsó 14 nap grafikon adatsor: `SELECT date, views, downloads FROM file_stats_daily WHERE file_id = ? ORDER BY date DESC LIMIT 14` .
- Legutóbbi események: `SELECT * FROM file_events WHERE file_id = ? ORDER BY id DESC LIMIT 25` (IP anonimization `anonymize_ip()` ha viewer nem admin).
- UI / output:
 - Grafikon (JS) a napi trendekhez, tábla az összesített mutatókhoz, eseménylista (anonimizált IP-k non-adminoknak).
- Biztonsági megfontolások:
 - Ellenőrizni kell, hogy a lekérdezés csak a feltöltőhöz tartozó fájlokra fut (auth és file.owner ellenőrzés).
 - Ne tegyen közzé nyers IP-címeket nem-adminok számára; használjon anonimizálást vagy CIDR / partial maskot.
 - Prepared statements minden DB lekérdezésnél; nagy aggregációk cache-elése javasolt.

4.11.22. messages (kiegészítés)

- Megjegyzés: ha van kliensoldali `assets/php/loadmessages.php` vagy `assets/js/script.js` -ben üzenetfrissítés, dokumentáljuk az API-t és polling/websocket viselkedést.

4.11.23. assets/php/db.php

- Fájl: [src/assets/php/db.php](#)
- Mi történik:
 - Létrehoz egy `mysqli` kapcsolatot: `$conn = new mysqli("localhost", "root", "", "jegyzetar");`
 - Ha kapcsolat hiba van: `die("Connection failed! " . $conn->connect_error);`
 - Egyszerű, központi connection bootstrap — a projekt többi fájlja ezt `require` -eli.

Megjegyzés: a kapcsolat beállításokat környezeti változókból vagy `.env` -ből érdemes kezelni (jelenleg hardcoded).

4.11.24. assets/php/functions.php

- Fájl: [src/assets/php/functions.php](#)
- Főbb exportált segédfüggvények és viselkedésük:
 - `db_log_error(mysqli $conn, string $message, ?string $sql = null, array $params = [])` : hibák logolása `error_log` és `src/assets/logs/db_errors.log` fájlba.
 - `db_stmt(mysqli $conn, string $sql, string $types = '', array $params = [])` : `mysqli_stmt` : prepared statement wrapper; validálja a `types` és `params` hosszát, bind-eli a paramétereket, végrehajtja és kivételt dob hibánál.
 - `db_query(mysqli $conn, string $sql, string $types = '', array $params = [])` : `mysqli_result` : lekérdezés wrapper, `db_stmt` -re épül, visszaadja a `mysqli_result` -ot.
 - `db_exec(mysqli $conn, string $sql, string $types = '', array $params = [])` : `bool` : végrehajtás wrapper (INSERT/UPDATE/DELETE), `true` visszatéréssel.

- `Message($text)` : egyszerű JS `alert()` wrapper (régí kimeneti mód, érdemes konszolidálni).
- `CodeGenerator()` : rövid alfanumerikus kód generátor (reg-kódokhoz használható).
- `t(string $key, string $fallback='')` : fordítási helper (globális `$translations` tömbre támaszkodik).
- IP és esemény naplózás: `get_client_ip()` , `log_file_event(mysqli $conn, int $fileId, ?int $userId, string $type, ?int $rating = null)` — `file_events` táblába ír, cooldown logika (alap 600s) és IP tárolás INET6_ATON-tal.
- Egyéb segédfüggvények: `anonymize_ip` , `fmt_event_label` , `strip_accents` , `tokenize_query` , `build_snippet` , `highlight_many_html` , `format_bytes` , `fav_star_row` , `fav_file_icon_svg` — keresés/sablonok és UI helper logika.

Biztonsági megjegyzések: `db_stmt` és társai prepared statements-t használnak és hibát dobnak logolással; `log_file_event` IP-kezelésnél INET6_ATON-t használ. `Message()` közvetlen alertet ír, érdemes átállítani strukturált üzenetkezelésre.

4.11.25. assets/php/group_actions.php

- Fájl: [src/assets/php/group_actions.php](#)
- Főbb műveletek (POST feldolgozás):
 - `join_group` — csatlakozás; ha a csoport privát → `status='pending'` , különben `status='accepted'` ; beszúr `group_members` rekordot.
 - `remove_member` — tulaj által távolíthat el tagot (`DELETE FROM group_members ...`).
 - `elfogadas` / `elutasitas` — pending beadványok elfogadása (`UPDATE ... SET status='accepted'`) vagy törlése.
 - `jegyzet_elfogadas` / `jegyzet_elutasitas` — a `group_files` jóváhagyása/eltávolítása (`UPDATE` / `DELETE`).
 - `uj_jegyzet` — csoportba feltöltés: feltöltött fájl mozgatása a `users/<username>/` mappába, majd `INSERT INTO group_files(..., is_approved)` (tulajtól függően azonnal elfogadott vagy pending).
 - `kilepes` — kilépés törléssel `group_members` -ből; `csoport_torles` — tulaj által csoport és kapcsolódó rekordok törlése (`group_members` , `group_files` , `groups`).

Megjegyzés a kódstílusról: a fájl több helyen `$_POST` mezőkre és `$_FILES` -ra épít és `$conn->query()` -t használ közvetlen SQL-sztringekkel (nem mindenhol prepared statement). Érdemes a bemeneteket sanitizálni és prepared statementeket használni a biztonság és stabilitás érdekében.

4.11.26. assets/php/group_init.php

- Fájl: [src/assets/php/group_init.php](#)
- Fő funkciók:
 - Kötelező `id` (GET) ellenőrzése; redirect/exit, ha hiányzik vagy nem található a csoport.
 - Betölti a csoport adatait (`SELECT * FROM groups WHERE id=?`) és beállítja: `$csoport_nev` , `$csoport_leiras` , `$tulaj_id` , `$privat` .
 - Lekérdezi az aktuális user tagsági állapotát a `group_members` táblából és beállít logikai flag-eket: `$aktualis_felhasznalo_tag` , `$aktualis_felhasznalo_tulaj` , `$aktualis_felhasznalo_pending` .

Használat: `group.php` include-olja ezt a file-t, hogy meg legyenek a csoport- és jogosultsági változók a renderhez.

4.11.27. assets/php/loadmessages.php

- Fájl: [src/assets/php/loadmessages.php](#)
- Viselkedés:
 - Várja a `GET[ 'friendid' ]` paramétert és a cookie-ból a saját `id` -t; `intval()` -lal castolja.
 - Lekérdezi a két felhasználó közötti üzeneteket: `SELECT * FROM messages WHERE (fromid=$userid AND toid=$friendid) OR (fromid=$friendid AND toid=$userid) ORDER BY sent_at ASC` .
 - Kliensnek HTML-kimenetet ad (saját/partner üzenetek eltérő osztállyal); `htmlspecialchars()` -t használ az üzenet tartalmára.
 - Támogatott egy `countonly=1` mód, ami csak a sorok számát írja ki.

Megjegyzés: a fájl egyszerű rendering endpointként működik — ha magas forgalmat várunk, érdemes AJAX/JSON API-t és paginációt bevezetni.

4.11.28. assets/php/download.php

- Fájl: [src/assets/php/download.php](#)
- Fő lépések:
 - Beolvassa `$_GET[ 'id' ]` -t (`intval`) és lekéri a `files` rekordot.
 - Lekéri a feltöltő `users` rekordját, majd felépíti a fájl elérési útját: `dirname(getcwd(), 2) . "/users/" . $user[ 'username' ] . "/" . $file[ 'file_name' ]` .
 - Ha létezik a fájl: beállítja a letöltéshez szükséges HTTP fejléceket (`Content-Disposition` , `Content-Length` , stb.) és `readfile()` -lal kiadja.
 - Ha nem található a fájl: egyszerű hibaüzenetet ír.

Biztonsági megjegyzés: nincs alkalmazva jogosultság-ellenőrzés (ki tölthet le egy fájlt), a path-összeállításnál érdemes canonicalizálni/validálni és relatív storage root-ot használni. A `dirname(getcwd(),2)` használata környezettől függően befolyásolja a path helyességét.

4.11.29. assets/php/report.php

- Fájl: [src/assets/php/report.php](#)
- Főbb viselkedés:
 - `require_once` -ok: `db.php` , `functions.php` , `lang.php` ; `session_start()` és reporter azonosítása cookie/session alapján.
 - Csak `POST` kérést fogad; bemenetek: `type` (`user|group|note`), `target_id` , `reason` , `redirect` .
 - Validál: típus engedélyezett-e, `target_id` numerikus-e, `redirect` biztonságos út-e (startsWith `/`).
 - Ellenőrzi, hogy nincs-e már nyitott jelentés ugyanattól a usertől (`SELECT id FROM reports WHERE reporter_id = ? AND target_type = ? AND target_id = ? AND status = 'open' LIMIT 1`) — prepared `db_query` hívással.
 - Beszúrja az új jelentést: `INSERT INTO reports (reporter_id, target_type, target_id, reason) VALUES (?, ?, ?, ?)` a `db_stmt` segítségével.

- Végül redirect a `redirect` mező értékére.

Megjegyzés: ez a fájl példás használata a `db_stmt` / `db_query` wrapper-eknek; validálja a bemeneteket és védi a duplikált open jelentéseket.

4.11.30. assets/php/premium.php

- Fájl: [src/assets/php/premium.php](#)
- Exportált függvények és viselkedés:
 - `premium_aktiv($premium_ig)` : visszaadja, hogy a `$premium_ig` (dátum string) még érvényes-e (`strtotime >= time()`).
 - `user_premium($conn, $felhasznalo_id)` : lekérdezi a `premium_users` táblából a `MAX(premium_ig)` értéket és meghívja `premium_aktiv` -ot; visszatérési érték boolean.
 - `premium_aktivalas_30nap($conn, $felhasznalo_id)` : ha létezik rekord a `premium_users` -ben → `UPDATE premium_ig = DATE_ADD(NOW(), INTERVAL 30 DAY)`, különben `INSERT` új rekordot.
 - `premium_datum($conn, $felhasznalo_id)` : visszaadja a `MAX(premium_ig)` -et stringként vagy `' '` -t, ha nincs.

Megjegyzés: a függvények általában `$conn->query()` -t használnak, a `felhasznalo_id` -t integerre castolják; érdemes prepared statementre és tranzakciós logikára váltani, ha a fizetési folyamatot éles integrációnál használjuk.

4.11.31. assets/php/accept_friend.php

- Fájl: [src/assets/php/accept_friend.php](#)
- Route / hely: AJAX vagy form POST feldolgozó, közvetlen include nélkül hívható a kliensről.
- Requires: `require "db.php"` a kapcsolatért.
- Auth / Input:
 - Ellenőrzés: cookie `id` megléte kötelező (`isset($_COOKIE['id'])`).
 - POST mező: `fromid` (a barátkérés küldőjének azonosítója; a fájl `intval` -lal castolja).
 - Ha hiányzik a cookie vagy a `fromid` , a script redirectel `../.. /index.php` -re.
- DB műveletek (kódból):
 - `UPDATE friends SET status = 1 WHERE fromid=$fromid AND toid=$myid` — a barátkérés elfogadása (a fájl `int` -re castolja a bemeneteket, de nem használ prepared statementet).
 - `SELECT username FROM users WHERE id=$fromid LIMIT 1` — lekéri a kérő felhasználó `username` -ét, hogy profilra lehessen redirectelni.
- Kimenet / redirect:
 - Siker esetén a kód redirectel `../.. /profile.php?user=<username>` -re; ha nincs username, redirect `../.. /notify.php` -ra.
- Fájrendszer, állapotváltozás:
 - Csak adatbázis-módosítás (`friends.status update`); nincs fájlművelet.
- Biztonsági megjegyzések és fejlesztői javaslatok:
 - A bemeneteket `intval` -lal castolják, így SQL injection kockázata alacsony, de a `db_stmt` / prepared statements használata konzisztensen javasolt.

- Nincs CSRF token ellenőrzés — érdemes POST-űrlapoknál CSRF-t bevezetni.
- Javasolt ellenőrizni, hogy egy nyitott `friends` sor tényleg létezik és `toid` egyezik a bejelentkezett user-rel a `UPDATE` előtt, és hogy a művelet nem enged visszaélésre.
- A redirect célját érdemes validálni és `urlencode()` használata megfelelő (a kód már `urlencode` -olja a username paramétert), továbbá limitálni a redirect útvonalakat relatív útvonalra.
- Érdemes értesítést (notify) létrehozni a kérő felhasználó részére az elfogadásról, ha a UI ezt megkívánja.

4.11.32. assets/php/add_friend.php

- Fájl: [src/assets/php/add_friend.php](#)
- Route / hely: POST feldolgozó endpoint, kliens által meghívott barátküldési művelet.
- Requires: `require "db.php"` a DB kapcsolatért.
- Auth / Input:
 - Cookie alapú auth: `$_COOKIE['id']` kötelező (küldő azonosítója).
 - POST mező: `toid` (a megcélzott felhasználó id-je; int-re castolódik).
 - Hiányzó cookie vagy `toid` esetén redirect `../../index.php`.
- DB műveletek (kódból):
 - Ellenőrzi, hogy a célfelhasználó létezik: `SELECT username FROM users WHERE id = $toid LIMIT 1`.
 - Duplikációs ellenőrzés: `SELECT * FROM friends WHERE (fromid=$fromid AND toid=$toid) OR (fromid=$toid AND toid=$fromid)` — ha nincs sor, beszúrja a barátkérést.
 - INSERT barátkérés: `INSERT INTO friends (fromid, toid, status) VALUES ($fromid, $toid, 0)` (status 0 = pending).
 - INSERT értesítés: `INSERT INTO notifys (fromid, toid, notifytype, readed) VALUES ($fromid, $toid, 'friend', 0)` — értesíti a címzettet.
- Kimenet / redirect:
 - Végül redirect `../../profile.php?user=<username>` (a célfelhasználó username-ével URL-enkódolva).
- Fájrendszer, állapotváltozás:
 - Csak adatbázis módosítások (friends, notifys), nincs fájlrendszer-művelet.
- Biztonsági megjegyzések és fejlesztői javaslatok:
 - A kód `int` -re castolja az inputokat, de nem használ prepared statementeket; javasolt a `db_stmt` / `db_query` wrapperrel prepared statements-re váltani.
 - Javasolt validálni, hogy a `fromid` és `toid` nem egyezik (ne küldhessen valaki magának barátkérést).
 - Ellenőrizni kell, hogy létezik-e már elfogadott kapcsolat (`status = 1`) és ennek megfelelően ne hozzon létre új bejegyzést.
 - Nincs CSRF védelem — be kell vezetni a POST űrlapoknál CSRF tokenet.
 - Érdemes logolni/szűrní a túlzott kéréseket (rate limit) és a spam elleni védelem alkalmazása.

4.11.33. assets/php/ads.php

- Fájl: [src/assets/php/ads.php](#)
- Mi történik:

- `require_once "premium.php"` -ot használja, és ha a felhasználó prémium (néma hirdetés), akkor a fájl egyszerűen visszatér (`return`) és nem ír ki semmit.
- Megkeresi az `assets/ads` mappában található képeket (`glob` a `__DIR__ . "/../ads"` mappában a `jpg, jpeg, png, webp` kiterjesztésekkel).
- Kiválaszt három véletlenszerű képet (bal, jobb, mobil) a megjelenítéshez és HTML `img` tageket echo-z a `assets/ads/<basename>` útvonalra hivatkozva.
- Input / Auth:
 - Nem fogad GET/POST bemenetet. Opcióként olvassa a cookie `id` -t a prémium ellenőrzéshez.
- Fájlrendszer / asset vonatkozások:
 - Elvárja, hogy a reklámképek fizikai fájljai a `src/assets/ads/` (projekthelyi) mappában legyenek; a kimeneti `src` URL `assets/ads/<file>` lesz, tehát a szervernek statikusan kiszolgálva kell legyen.
 - Nem ír adatbázisba, csak fájlrendszeri olvasást végez (`glob`).
- Biztonsági és fejlesztői megjegyzések:
 - A képek `basename()` -jét használja a `src` attribútumban; érdemes `htmlspecialchars()` -szal escape-elni a kiírt `src` és `alt` attribútumokat, bár a fájlnev forrása a szerveren lévő fájlok listája.
 - Gondoskodjunk arról, hogy a `assets/ads` mappa ne tartalmazzon nem kívánt fájlokat; ha a mappába támadható tartalom kerülhet, további validáció szükséges.
 - Mivel a fájl a kliensnek közvetlen HTML-t ad vissza, ellenőrizzük a responsive viselkedést és a lazy-loading használatát nagyobb teljesítményért.
 - A prémium logikát auditálni kell: `user_premium()` meghívása használja a `premium.php` segédfüggvényét — győződjünk meg róla, hogy a helper hatékony cache-eléssel működik, különösen sokfelhasználós oldalnál.

4.11.34. assets/php/delete.php

- Fájl: [src/assets/php/delete.php](#)
- Mi történik:
 - Betölti a DB kapcsolatot `require "db.php"` -vel, majd a `$_COOKIE['id']` alapján lekéri a felhasználó sorát (`SELECT * FROM users WHERE id='...'`) .
 - Ha GET `id` paraméter érkezik, lekérdezi a `files` rekordot (`SELECT * FROM files WHERE id='...'`). Ha létezik, törli a `files` rekordot (`DELETE FROM files WHERE id='...'`) és megpróbálja törölni a fájlt a fájlrendszerről (`unlink($path)`).
 - A kód egy `unlink($tn_path)` hívást is tartalmaz, de a `$tn_path` nincs sehol definiálva a fájlban (potenciális hibaforrás).
- Bemenetek / Auth:
 - Cookie `id` szükséges (a kód közvetlenül használja `$_COOKIE['id']` anélkül, hogy létezését ellenőrizné robust módon).
 - GET paraméter: `id` — a törlendő fájl azonosítója.
- Fájl- és DB-műveletek / Következmények:
 - SQL SELECT/DELETE a `files` táblában, majd fizikai törlés a szerveren: a path `getcwd()` . `"\\assets\\users\\" . $user['username'] . "\\" . $file_name` alapján épül.
 - Ha a fájl nem található, a kód hibaüzenetet ír ki; siker után redirect `myprofile.php` .

- Hibák / problémák a jelenlegi implementációban:
 - SQL injection: a lekérdezések string-konkatenációval készülnek (`"... WHERE id='$file_id'"`), nem használnak prepared statementeket.
 - Auth/ownership nincs ellenőrizve: nem biztosítják, hogy a bejelentkezett user valóban a fájl tulajdonosa vagy admin; ez lehetővé teszi jogosulatlan törlést ha a cookie manipulálható.
 - A kód közvetlenül használja `$_COOKIE['id']` anélkül, hogy `isset()` -tel vagy típusellenőrzéssel védené (ez hibához vezethet, ha nincs cookie).
 - `$tn_path` nincs definiálva mielőtt `unlink($tn_path)` -t hívnának — runtime warning / error.
 - Path-összeállítás Windows-specifikus backslash-ekkel és `getcwd()` -vel; a projekt többi része relatív `users/<username>/` utat használ, érdemes következetes storage-root konstans használata.
 - Nincs CSRF védelem, és destruktív művelet GET paraméterrel történik (jobb lenne POST + CSRF + explicit confirm).
- Fejlesztői javaslatok (ajánlott javítások):
 - Használjon prepared statements (`db_stmt`) minden DB művelethez és validálja a `file_id` -t `intval()` -lal.
 - Ellenőrizze, hogy a bejelentkezett user (`$_COOKIE['id']`) megegyezik-e a `files.uploaded_by` értékével, vagy hogy a user admin-e, mielőtt törlést engedélyez.
 - Definiálja és validálja a thumbnail/tn-path változót (`$tn_path`) mielőtt `unlink` -olná.
 - Váltson POST-alapú törlésre, és vezessen be CSRF tokenet, valamint egy szerver-oldali confirm/check lépést.
 - Canonicalizálja a fájlvonalat és használjon egy állandó `STORAGE_ROOT` -ot, például a projekt gyökere felé mutató abszolút elérési utat, hogy path traversal se legyen lehetséges.
 - Logolja az admin/fájl törléseket audit táblába (`deletions`), és ne írjon részleteket publikusan hibauzenetekbe.

4.11.35. assets/php/findanything.php

- Fájl: [src/assets/php/findanything.php](#)
- Mi történik:
 - GET paraméterek alapján (`keresett` , opcionálisan `rating`) végez fájl- és felhasználó-keresést, és HTML `search-card` elemeket render ki (kártyák és formok).
 - Direktbe `echo` -z HTML-t a válaszként, nem JSON API-ként működik.
- Bemenetek / Auth:
 - GET `keresett` (keresőszó, `htmlspecialchars()` + `trim()` által feldolgozva).
 - GET `rating` (opcionális, szűrőként, `intval()` castolódik).
 - Cookie `id` opcionálisan (ha létezik, a felhasználó saját magát kizárja a user-keresés eredményeiből).
- DB műveletek (kódból):
 - Fájlok: `SELECT * FROM files WHERE (name LIKE '%...' OR subject LIKE '%...' OR tags LIKE '%...') + opcionális AND rating = ?` .
 - Felhasználók: `SELECT * FROM users WHERE username LIKE '%...' AND id != <loggedInUserId>` .
 - Friend ellenőrzés: `SELECT * FROM friends WHERE (fromid = ? AND toid = ?) OR (fromid = ? AND toid = ?)` .

- Kimenet / Render:
 - Fájl kártyák: `<div class="search-card">` név és link-gomb a `note.php?id=<file_id>` -re.
 - User kártyák: form `add_friend.php` -re, "Jelölés" gomb vagy "Már barátok vagytok" szöveg.
 - Ha nincs keresési feltétel, a kód korai exit-tel `<p>Kezdj el gépelni...</p>` üzenettel.
- Biztonsági megjegyzések és fejlesztői javaslatok:
 - A `keresett` bemenet `real_escape_string()` -gel van feldolgozva a `LIKE` záradékban, de egyéb lekérdezésekre nem — javasolt prepared statements (`db_query` / `db_stmt` wrapperek) használata az összes lekérdezéshez.
 - A `rating` lekérdezés `intval()` castolódik (jó), de a fájl szűrésére nem értékek lekérése történik, csak `LIKE` keresés van.
 - A kód közvetlenül HTML-t echo-z; XSS-re veszélyes ha a `file['name']` vagy `user['username']` tartalmazhat injected HTML-t — a `htmlspecialchars()` használata jelenlegi, de érdemes összes felhasználói mező outputját és attribútumait escape-elni.
 - Az `$sqlFriendCheck` nem escapeelt lekérdezés — javasolt prepared statement vagy `real_escape_string()` az összes paraméterhez.
 - Nincs rate limiting — sok keresési kérés potenciálisan teljesítményt befolyásolhatnak; javasolt lekérdezések cache-elése vagy pagination.
 - Az endpoint AJAX-szerű autocomplete-hez használható, de a kód nem kezeli az AJAX header-eket vagy JSON-t, csak HTML-t render ki — a frontend feladata feldolgozni a HTML és beilleszteni a DOM-ba.

4.11.36. assets/php/findtag.php

- Fájl: <src/assets/php/findtag.php>
- Mi történik:
 - GET paraméterre alapján (`keresett` tag szöveg) lekérdezi és listázza a `tags` tábla egyezéseit.
 - Direktbe HTML `div` elemeket echo-z a tag előnézetre.
- Bemenetek / Auth:
 - GET `keresett` — keresendő tag string (nincs escape-elés vagy prepared statement).
- DB műveletek (kódból):
 - `SELECT * FROM tags WHERE tags LIKE '%$keresett%'` — **SQL injection kockázat:** a `$keresett` bemenet közvetlenül beágyazódik a lekérdezésbe escaped-statement nélkül.
- Kimenet / Render:
 - `<div>` -ek és link-ek: `<a href="..." onclick="...">` a tag kiválasztásához.
- Biztonsági problémák és fejlesztői javaslatok:
 - **KRITIKUS:** az SQL lekérdezésString interpolációval épül fel — szükséges prepared statement-re váltani (`db_query` wrapper használatával és parameterezett lekérdezéssel).
 - A HTML output sem szükségképpen escape-elt, XSS kockázat is lehetséges — `htmlspecialchars()` -szal kell escape-elni az outputot.
 - Rate limiting ajánlott az autocomplete kérésekhez.

4.11.37. assets/php/footer.php

- Fájl: <src/assets/php/footer.php>

- Mi történik:
 - Statikus footer HTML render, amely magában foglalja a copyright, fejlesztői linkeket és jogi linkeket (Privacy, Terms).
 - Fordítási helpereket (`t()`) használ a footer szövegekhez.
- Bemenetek / Auth:
 - Nincs GET/POST bemenet; olvasható file, amely include-ként fut.
- Include / Import:
 - `require "lang.php"` — fordítási rendszer az üzenetekhez.
- Kimenet / Render:
 - HTML `<footer>` tag a footer navigációval és linkekkel.
- Biztonsági megjegyzések:
 - Statikus szöveg, de fordítási kulcsok (`t('footer_copyright')` stb.) biztosítják a lokalizációt.
 - Fontos, hogy a fordítási kulcsok biztonságosan kezeltenek legyenek a `lang.php` -ban (semmilyen user input nincs itten közvetlen felhasználva).

4.11.38. assets/php/kereso_tag.php

- Fájl: src/assets/php/kereso_tag.php
- Mi történik:
 - jQuery-alapú kliensoldali tag autocomplete widget — a bemenet mezőt köti a `findtag.php` AJAX híváshoz.
 - Az autocomplete lista HTML-t (div-eket) megjelenít a kiválasztott tag append-jeléhez az `#tag` input mezőbe.
- Bemenetek / Auth:
 - Klienoldali: `#kereso` mezőre gépelés -> AJAX POST/GET a `findtag.php` -ra -> autocomplete lista append.
- Kimenet / Render:
 - jQuery trigger-ek, `#taglist` divbe append-elt `<div>` elemek, onclick event handler a kiválasztáshoz.
- Biztonsági megjegyzések:
 - A kliensoldali HTML append-elés során a `findtag.php` -ból érkezett HTML beillesztésre kerül — szükséges, hogy a `findtag.php` outputja XSS-mentes legyen, tehát `htmlspecialchars()` -sal escape-elt legyen a tag szöveg.
 - Az AJAX kérésnek nincs CSRF protection — javasolt bevezetni.

4.11.39. assets/php/lang.php

- Fájl: <src/assets/php/lang.php>
- Mi történik:
 - Központi lokalizációs rendszer: lekérdezi a `translations` táblát és betölti a `$GLOBALS['translations']` tömbbe a megadott nyelvhez.
 - A `t()` segédfüggvényt definiálja, amely a fordítás-tömbből végzi a kulcsok felkeresését és fallback szöveget biztosít.

- Session/cookie alapú nyelvválasztás: ha nincs `$_SESSION['lang']` vagy `$_COOKIE['lang']`, alapértelmezés a `'hu'`.
- Bemenetek / Auth:
 - GET/POST `lang` paraméter — whitelist validációval (`hu|en|de`).
 - Session / Cookie `lang` — ugyancsak whitelist.
- DB műveletek (kódból):
 - `SELECT * FROM translations WHERE lang_code = ?` (prepared statement via `db_query` — **BIZTONSÁGOS**).
 - Missing-key seeding: ha egy kulcs nem létezik a fordításban, automatikus `INSERT INTO translations (t_key, lang_code, t_value) VALUES (?, ?, ?)` az összes támogatott nyelvhez (ha az "auto-seed" opció aktív).
- Kimenet / Render:
 - Globális `$translations` tömb feltöltése, `t()` függvény export, session/cookie update a megválasztott nyelvhez.
- Biztonsági megjegyzések:
 - **JÓ GYAKORLAT:** a `lang` paraméter whitelist-tel van validálva (`in_array($lang, ['hu', 'en', 'de'])`), így SQL injection nem lehetséges ebben az ágban.
 - A `db_query` wrapper-t használja (prepared statement), amely biztonságos.
 - Az auto-seeding logika gondoskodik arról, hogy a hiányzó fordítási kulcsok bekerüljenek az adatbázisba — hasznos fejlesztéshez, de éles szinten monitoring javasolt, hogy véletlenül ne kerüljenek be debug/teszt kulcsok.

4.11.40. assets/php/logout.php

- Fájl: [src/assets/php/logout.php](#)
- Mi történik:
 - Egyszerű logout logika: invalidálja a cookie-t (érték törlése + lejáratí idő múltban) és session undef-elése.
- Bemenetek / Auth:
 - Cookie `id` — törlendő. Nincs POST validáció szükséges.
- Kimenet / Redirect:
 - `setcookie("id", "", time() - 3600, "/", $_SERVER["HTTP_HOST"] ?? "", true, true)` — cookie törlés HttpOnly és Secure flag-gel (jó).
 - `header("Location: index.php")` — redirect.
- Biztonsági megjegyzések:
 - **JAVASLAT:** `session_start()` és `session_destroy()` is javasolt, ha sessiont használnak erre az alkalmazásra (mostani kód csak a cookie-t kezeli, ami elég lehet a cookie-alapú auth-hoz).
 - A HttpOnly és Secure flag-ek helyesen beállítódnak.
 - Opcionál: logout eseményt lehetne logolni az `event_logs` táblába az audit céljára.

4.11.41. assets/php/mail-2fa.php

- Fájl: [src/assets/php/mail-2fa.php](#)

- Mi történik:
 - Kétlépcsős hitelesítési kód generálása, e-mailben küldése és a `2fa_codes` táblába mentése.
 - A kód jellemzően az `reglog.php` -ből hívódik meg a login folyamat 2. lépcsőjeként.
- Bemenetek / Auth:
 - `Session id` és `email` szükséges (a bejelentkezett felhasználó).
 - Az `$conn` DB kapcsolat szükséges (include vagy paraméter).
- Generálás és küldés (kódból):
 - `random_int(10000, 99999)` — 5-jegyű numerikus kód.
 - `mail($email, $subject, $body, $headers)` — PHP beépített mail() függvénnyel küldés.
 - `INSERT INTO 2fa_codes (userid, code, created_at, expires_at) VALUES (?, ?, NOW(), DATE_ADD(NOW(), INTERVAL 10 MINUTE))` (via `db_exec` / prepared statement).
- Kimenet / Render:
 - E-mail szöveg: a kód és belépési linkkel; lokalizáció via `t()` .
- Biztonsági megjegyzések:
 - A kód csak 5 számjegy (100000 kombinációs lehetőség) — még elfogadható korlátolt próbálkozás mellett, de 6-jegyűre növelés javasolt.
 - **FONTOS:** Az `mail()` PHP függvény használata nem biztosít tls/smtp auth-ot — javasolt SMTP library (Swift Mailer, PHPMailer) használata validált szenderre.
 - A kód 10 percig érvényes (`expires_at`), ami ésszerű.
 - Rate limiting a `mail()` hívásokra (pl. max 5 kód/nap user-ként) javasolt a spam/abuse elleni védelemhez.

4.11.42. assets/php/mail-regver.php

- Fájl: [src/assets/php/mail-regver.php](#)
- Mi történik:
 - Regisztrációs aktiválási token generálása és e-mailben küldése; a token a `tokens` táblában tárolódik.
 - A kód az `reglog.php` -ből hívódik meg sikeres regisztráció után.
- Bemenetek / Auth:
 - `Session ver_id` (új user ID) és `email` szükséges.
 - Az `$conn` DB kapcsolat.
- Token generálás és küldés (kódból):
 - `random_int(100000, 999999)` — 6-jegyű numerikus token.
 - `INSERT INTO tokens (userid, token, created_at, expires_at) VALUES (?, ?, NOW(), DATE_ADD(NOW(), INTERVAL 1 DAY))` (prepared statement).
 - E-mail linkben az aktiváció: `http://localhost/.../reg-ver.php?token=<token>` — **PROBLÉMA:** hardcoded `localhost` használata, éles környezetben `$_SERVER['HTTP_HOST']` vagy konfigurációs domain szükséges.
- Kimenet / Render:
 - E-mail szöveg: aktiválási link a tokennel.
- Biztonsági megjegyzések:

- **KRITIKUS:** a hardcoded `localhost` a linken — szükséges dinamikusra cserélni: `$_SERVER['HTTP_HOST']` vagy `$_SERVER['HTTP_X_FORWARDED_HOST']` (proxy esetén).
- A token 1 napig érvényes, ami ésszerű.
- Az `mail()` függvény korlátozásai ugyanazok, mint a `mail-2fa.php`-nál — javasolt SMTP library.
- Rate limiting és token reuse-védelem (`tokens` tábla státusza) javasolt.

4.11.43. assets/php/navbar.php

- Fájl: [src/assets/php/navbar.php](#)
- Mi történik:
 - Fejléc navigációs sáv renderelése; dinamikus, attól függően, hogy bejelentkezett-e a felhasználó vagy sem.
 - Linkek: főoldal, csoportok, mentett Jegyzetek (kedvencek), üzenetek, profil, kijelentkezés (ha bejelentkezve); bejelentkezés/regisztráció (ha nem).
 - Fordítási segédfüggvényt (`t()`) használ a navbar szövegekhez.
- Bemenetek / Auth:
 - Cookie `id` ellenőrzése (if isset és numerikus).
 - Az `$translations` globális tömb a `lang.php`-ból.
- Include / Import:
 - `require_once "lang.php"` vagy include a navbar renderhez szükséges.
- Kimenet / Render:
 - HTML `<nav>` tag Bootstrap/CSS osztályokkal (pl. `navbar navbar-expand-lg`), links és user dropdown.
 - Értesítési badge a nem olvasott üzenetekhez (ha van `$unread_count` közvetítve).
- Biztonsági megjegyzések:
 - A dinamikus mező (felhasználónév, értesítés szám) escape-elve kell legyen: `htmlspecialchars($username)`.
 - Az `id` cookie felhasználódik az értesítés lekérdezéshez — a tényleges lekérdezés `SELECT COUNT(*) FROM notifys WHERE toid = ? AND readed = 0` javasolt hogy a navbar-ben vagy a fetch-előtt megtörténjen az auth validáció.
 - A navbar sok oldalon include-ódik (`index.php` , `upload.php` , stb.), ezért a fájl biztonsága kritikus.